TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

Nguyễn Văn A - Trần Thị B

HƯỚNG DẪN TRÌNH BÀY BÁO CÁO KHÓA LUẬN TỐT NGHIỆP/ THỰC TẬP TỐT NGHIỆP/ THỰC TẬP DỰ ÁN TỐT NGHIỆP

KHÓA LUẬN TỐT NGHIỆP CỬ NHÂN CNTT
CHƯƠNG TRÌNH CHUẨN

Tp. Hồ Chí Minh, tháng MM/YYYY

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

Nguyễn Văn A - 1512090
Trần Thị B - 1512007

HƯỚNG DẪN TRÌNH BÀY BÁO CÁO KHÓA LUẬN TỐT NGHIỆP/ THỰC TẬP TỐT NGHIỆP/ THỰC TẬP DỰ ÁN TỐT NGHIỆP

KHÓA LUẬN TỐT NGHIỆP CỬ NHÂN CNTT
CHƯƠNG TRÌNH CHUẨN

GIẢNG VIÊN HƯỚNG DẪN
PGS.TS. Hoàng Văn C
TS. Lê Thị D

Tp. Hồ Chí Minh, tháng MM/YYYY

Lời cam đoan

Tôi xin cam đoan đây là công trình nghiên cứu của tôi dưới sự hướng dẫn của ...

Thuyết minh chỉnh sửa đề tài

Bản thuyết minh chỉnh sửa đề tài (nếu có thay đổi tên đề tài, nội dung báo cáo, . . . trong cuốn báo cáo sau bảo vệ)

Nhận xét hướng dẫn

Bản nhận xét của giảng viên hướng dẫn (có chữ ký) do giáo vụ cung cấp.

Nhận xét phản biện

Bản nhận xét của giảng viên phản biện (có chữ ký) do giáo vụ cung cấp.

Lời cảm ơn

Tôi xin chân thành cảm ơn ...

Đề cương chi tiết

Bản đề cương đã thực hiện và nộp cho giáo vụ trước đó *(phải có chữ ký của giảng viên hướng dẫn).*

Contents

List of figures

List of tables

Abstract

Giới hạn 200-300 từ, sử dụng văn phong học thuật, ngắn gọn, khách quan, viết liền mạch, không gạch đầu dòng.

Hướng dẫn trình bày tóm tắt:

1. Giới thiệu vấn đề/ngữ cảnh của vấn đề (khoảng 1 - 3 câu): Trình bày bối cảnh và vấn đề thực tế đang tồn tại/ khoảng trống nghiên cứu, tầm quan trọng của vấn đề nghiên cứu, lý do cần thực hiện đề tài.

2. Mục tiêu đề tài (1 câu): nêu cụ thể đề tài giải quyết/phát triển điều gì.

3. Phương pháp và giải pháp (1 - 3 câu): Trình bày phương pháp, công nghệ, công cụ, mô hình, giải pháp được đề xuất trong đề tài. Nêu những cải tiến nổi bật (nếu có). Nêu phương pháp thử nghiệm, đánh giá, bộ dữ liệu được sử dụng để đánh giá (nếu có)

4. Kết quả chính (1 - 2 câu): Trình bày ngắn gọn các kết quả nổi bật, có thể đưa số liệu cụ thể (độ chính xác, tốc độ xử lý, dung lượng dữ liệu, . . .)

5. Kết luận (1 - 2 câu): Nêu ý nghĩa khoa học và thực tiễn (nếu có) của phương pháp/giải pháp hay kết quả của đề tài, khả năng áp dụng vào thực tế/đóng góp cho doanh nghiệp/người dùng/nghiên cứu sau này, . . .

Chapter 1

Introduction

1.1 Growth of cloud computing

The paradigm of cloud computing presents the fact that it is fulfilling a long-held dream in the information technology industry: computing as a utility [1]. This transformation has reshaped the way IT hardware is designed, purchased, and utilized, reconstructing the industry from a large up-front capital to a pay-as-you-go economic model. Historically, the growth of cloud computing was observed by the development of large-scale commodity-computer data centers, allowing providers to achieve significant economies of scale in electricity, hardware, and operations [1].

In addition, [1] shows three new aspects that mark the initial emergence of cloud adoption. Those are the illusion of infinite computing resources available on demand, the elimination of up-front commitments by users, and the ability to pay for resources on a short-term, as-needed basis. Thus, these aspects give cloud computing elasticity, which allowed organizations to attach resources to workloads much more closely, transferring the risks of overprovisioning and underprovisioning from the service operator to the cloud vendor [1]. A notable early example of this growth was the Animoto service, which scaled from 50 to 3,500 servers in three days after it was deployed on Facebook.

Furthermore, the growth of cloud computing is characterized by a de-

centralized infrastructure through emerging architectures such as serverless computing and edge computing [2]. Serverless architectures (Function-as-a-Service) allow developers to concentrate solely on application logic while the platform manages all underlying scaling, fault tolerance, and back-end management. Simultaneously, the increase in Internet of Things (IoT) devices has driven the adoption of edge and fog computing, bringing processing power closer to data sources to meet ultra-low-latency requirements and reduce network bottlenecks [2]. This continuous expansion into distributed, heterogeneous, and ad hoc environments highlights the increasing complexity of the modern cloud ecosystem.

As technology developed, the landscape evolved from single-provider environments to more complex infrastructures [2]. Today, hybrid cloud and multi-cloud models have become the dominant standard. [2], [3] indicate that approximately 73% of organizations now employ a hybrid cloud approach, combining public and private environments to balance cost, performance, and governance. Large enterprises, in particular, have observed a significant increase in cloud spending, with more than 76% of large organizations spending more than $5 million monthly on public cloud services by 2026. However, as these hybrid cloud architectures grow in sophistication and monthly spend, they require centralized oversight through a Cloud Center of Excellence (CCOE) or FinOps teams to manage the growing complexity and associated security risks [3]. Consequently, operational and security processes are required for hybrid cloud infrastructure management.

1.2 Emergence of hybrid cloud

In recent years, hybrid cloud has been widely adopted, transforming operations in many enterprise IT. As a result, this transformation helps organizations to be capable of integrating private and public cloud infrastructures into a single computing environment [4], [5]. This deployment

model allows enterprises to optimize resource utilization, improve business continuity, and maintain diverse regulatory and operational requirements while ensuring the flexibility to successfully deploy workloads across heterogeneous infrastructures. Moreover, [5] defines a base hybrid cloud that facilitates portability and interoperability of the application between multiple cloud environments. In this way, organizations can benefit from the scalability and elasticity of public cloud services without giving up control over sensitive data and mission-critical applications.

However, the integration of multiple cloud environments significantly increases the complexity of infrastructure management [6], [7], [5]. In general, hybrid cloud models consist of heterogeneous technologies, distributed resources, and multiple security models that must operate as a cohesive system. Consequently, organizations are required to maintain consistent governance, identity and access management, and security policies in both private and public cloud in hybrid cloud. As cloud-native applications and distributed services continue to scale, ensuring operational consistency and maintaining a security standard become increasingly difficult [8].

Furthermore, hybrid cloud environments require strict coordination between infrastructure management, application deployment, and security operations [6], [8]. Traditional security approaches, which are based on periodic assessments or isolated security modules, are incapable of addressing the dynamic nature of hybrid cloud infrastructures. Instead, security controls must be integrated throughout the system lifecycle to support continuous monitoring, automated vulnerability assessment, and policy enforcement. Consequently, organizations require operational processes that are integrated with automation, infrastructure management, and continuous security practices to effectively manage hybrid cloud environments while maintaining security, and compliance [7].

1.3 Research challenges

Although hybrid cloud provides flexibility and scalability, it also introduces some significant challenges in managing secure delivery and infrastructure operations. In addition, the hybrid cloud requires organizations to reconcile software deployment, security policies, and compliance while maintaining system performance. These challenges are identified as follows.

Challenge 1: Managing heterogeneous infrastructure. Hybrid cloud refers to multiple diverse computing platforms, including on-premises infrastructure, private cloud, and public cloud, where each one has different architectures, interfaces, and security models. Consequently, the transition from the traditional cloud model to the hybrid cloud model requires maintaining consistent operational procedures and configuration policies, which is considered to be complex and time-consuming. [5] also states that the hybrid cloud needs interoperability and portability among many distinct cloud infrastructures while preserving their individual characteristics. As a result, operational teams must manually manage diverse infrastructures that often consist of different methods, mechanisms, and security policies.

Moreover, [7], [8] agree that as organizations expand their models, the complexity of models escalates, increasing administrative overhead and the likelihood of security misconfigurations. This, by far, makes consistency across heterogeneous infrastructure increasingly difficult. [8] highlights that the increasing complexity in cloud infrastructure has become one of the primary concerns for most organizations. In addition, differences in configuration policies, identity management, and resource allocation increase administrative overhead and configuration inconsistencies that may expose security vulnerabilities.

Challenge 2: Integrating continuous security into the software lifecycle. Traditional security operations are usually performed during the later stages of software development, making vulnerabilities more expensive

and difficult to detect and remediate. Although this approach is capable of maintaining security standards for conventional software development models, it may expose serious vulnerabilities in modern cloud-native applications, which are continuously developed and deployed through automated CI/CD pipelines. Thus, [9] emphasizes that security operations should be continuously incorporated into software development and deployment processes to reduce software vulnerabilities and improve software integrity.

In hybrid cloud infrastructure, applications can be deployed across multiple infrastructures with different security requirements. Therefore, Security operations must perform not only on application source code but also on software dependencies, configuration policies, and resource allocation. Performing these assessments independently or only before production deployment increases the risk that vulnerabilities remain undetected until later stages, where remediation is more expensive and difficult.

Furthermore, organizations must integrate automated security operations into existing development workflows without interfering with development productivity. Additionally, Continuous vulnerability scanning and policy validation should operate alongside continuous integration and deployment processes while maintaining acceptable execution time. Consequently, many organizations try to balance between rapid software delivery and continuous security in hybrid cloud infrastructure [9].

Challenge 3: Assessing security findings and prioritizing risks. Modern software applications are usually developed with multiple security evaluation tools. These include static application security testing, software composition analysis, dependency vulnerability scanners, and infrastructure configuration analysis tools. Each tool focuses on detecting different categories of vulnerabilities and produces systematic reports with its own severity metrics and confidence levels. Although these tools cover each security category, they also generate diverse security findings that are dif-

ficult to evaluate consistently and prioritize risks.

In practice, security engineers manually analyze findings from multiple scanners, eliminate duplicate reports, and compare vulnerabilities to decide multiple actions for each risk. However, the manual process becomes more challenging as project size grows, which can slow down the whole deployment. In hybrid cloud infrastructures, where applications are updated and deployed frequently on multiple clouds, delayed or inconsistent prioritization may leave critical vulnerabilities unresolved while less significant issues receive unnecessary attention.

[9] recommends integrating security operations into a software development process to support continuous vulnerability management and prioritization. Nevertheless, effectively combining many different security findings with distinct severity metrics into meaningful risk assessments remains a significant challenge. Thus, organizations require systematic approaches capable of combining multiple security reports and prioritizing vulnerabilities according to their overall operational risk.

Challenge 4:Establishing an integrated SysSecOps process. The complexity of hybrid cloud environments requires close contact between infrastructure management, software delivery, and security operations. However, these activities are performed by separate teams using independent tools and workflows. Specifically, infrastructure administrators concentrate primarily on provisioning and maintaining computing resources, development teams emphasize rapid software delivery, while security teams can only perform security assessments independently after implementation. Consequently, these different processes create communication gaps, which delay security reports and reduce the overall efficiency of software delivery processes.

Although DevOps has significantly reduced the communication gaps between development and security teams, security assessments remain inconsistent across many organizations. Moreover, security operations are

usually known as isolated scanning stages rather than as continuous operational processes from infrastructure management, software development, deployment, to maintenance. As a consequence, as hybrid cloud infrastructures continue to grow in scale and complexity, fragmented security operations become increasingly difficult to manage while maintaining consistent security policies and regulatory compliance [8], [5].

1.4 Research gaps

Despite the rapid development of cloud computing technologies and the increasing adoption of DevSecOps practices, existing research has primarily focused on individual technologies, security mechanisms, or specific stages of the software development lifecycle. Multiple literature reviews consistently report that the current body of research remains fragmented across several dimensions, including security automation, tool integration, operational workflows, and empirical validation. Myrbakken and Colomo-Palacios observed that DevSecOps research predominantly concentrates on practices and tools rather than comprehensive operational frameworks [10]. Similarly, Rajapakse et al. identified continuous security integration, automation, and security orchestration as persistent research challenges through a systematic review of DevSecOps studies [11]. More recently, Zhao et al. concluded that opportunities remain for broader operational integration and more comprehensive validation of DevSecOps approaches across complex computing environments [12]. These observations suggest that several important research gaps remain, particularly for security operations within hybrid cloud environments.

Gap 1: Limited Operational Frameworks for Hybrid Cloud Security Existing research has proposed numerous DevSecOps practices, cloud security frameworks, and automated security solutions to improve secure software delivery. However, these studies primarily focus on software development

activities, security tools, or individual operational practices instead of providing an integrated operational process capable of coordinating infrastructure management, software deployment, and continuous security within hybrid cloud environments. While the NIST Cloud Computing Reference Architecture defines the architectural relationships among cloud components and describes the characteristics of hybrid cloud infrastructures, it does not prescribe an operational process for integrating system administration, software engineering, and security activities [5]. Likewise, both Myrbakken and Colomo-Palacios and Zhao et al. emphasized that existing DevSecOps research is largely organized around practices, tools, and lifecycle activities, whereas comprehensive operational frameworks remain comparatively limited [10], [12]. This indicates a need for research that investigates integrated operational processes tailored specifically to hybrid cloud environments.

Gap 2: Insufficient Integration of Continuous Security Throughout Hybrid Cloud Operations Continuous security has become a fundamental principle of secure software engineering, and the NIST Secure Software Development Framework recommends integrating security practices throughout software development and deployment activities to improve software integrity and reduce vulnerabilities [9]. Nevertheless, implementing continuous security remains a persistent challenge in practice. Rajapakse et al. identified automation of security activities, continuous security assessment, and balancing software delivery speed with security assurance as recurring challenges across existing DevSecOps studies [11]. Similarly, Myrbakken and Colomo-Palacios reported that security integration across the software lifecycle remains one of the central themes requiring further investigation [10]. While current research predominantly concentrates on integrating security into CI/CD pipelines, comparatively fewer studies investigate how continuous security should be coordinated across both software engineering activities and infrastructure operations in heterogeneous hybrid cloud

environments. Consequently, further research is required to establish operational processes that embed continuous security throughout the entire hybrid cloud operational lifecycle.

Gap 3: Limited Orchestration of Heterogeneous Security Tools and Security Findings Modern software projects increasingly employ multiple security assessment tools, including static application security testing, software composition analysis, dependency vulnerability scanning, and infrastructure security analysis. Although these tools improve vulnerability detection from different perspectives, they typically operate independently and generate heterogeneous outputs with different reporting formats, severity classifications, and confidence levels. Zhao et al. observed that existing DevSecOps literature primarily categorizes research according to security practices, tools, and metrics, while comparatively less attention has been devoted to the orchestration and integration of heterogeneous security tools into unified operational workflows [12]. Rajapakse et al. likewise identified tool integration and security automation as recurring research challenges affecting DevSecOps adoption [11]. These findings suggest that systematic approaches for coordinating multiple security tools, consolidating their outputs, and supporting unified operational decision-making remain relatively underexplored.

Gap 4: Limited Empirical Validation of Integrated Security Operational Processes Existing literature has proposed numerous recommendations, conceptual architectures, and best practices for implementing DevSecOps. However, comprehensive empirical validation of integrated operational processes remains comparatively limited. Myrbakken and Colomo-Palacios highlighted that much of the current literature discusses DevSecOps from conceptual or practical perspectives without providing comprehensive evaluation of integrated operational frameworks [10]. More recently, Zhao et al. identified empirical validation as one of the research dimensions requiring further attention, particularly for studies involving broader operational

integration and practical implementation [12]. Furthermore, Rajapakse et al. emphasized the need for additional empirical studies to evaluate integrated DevSecOps approaches in realistic operational environments [11]. Therefore, there remains a need for practical implementations that demonstrate how integrated security operational processes can be deployed and evaluated within hybrid cloud environments.

1.5 Research objectives

Based on the research challenges and gaps identified in the previous sections, the primary objective of this research is to design and develop a SysSecOps operational process for hybrid cloud environments that integrates system administration, continuous security, and software delivery into a unified workflow. The proposed process aims to improve the coordination between infrastructure management, security operations, and software development while supporting secure, automated, and efficient software deployment across heterogeneous cloud infrastructures.

To achieve this objective, the research is conducted with the following specific objectives:

- To investigate the operational and security challenges associated with hybrid cloud environments and analyze the limitations of existing approaches in integrating system administration, software development, and continuous security.

- To design a SysSecOps operational process that incorporates continuous security throughout the software operational lifecycle by integrating infrastructure management, automated security assessment, policy enforcement, and deployment activities into a unified workflow.

- To develop a prototype SysSecOps system that implements the pro-

posed operational process by integrating multiple security assessment tools, infrastructure automation, and continuous security mechanisms within a hybrid cloud environment.

- To establish a unified security assessment mechanism capable of aggregating security findings from heterogeneous security tools and supporting risk evaluation to facilitate security analysis and remediation prioritization.

- To evaluate the proposed SysSecOps system through representative hybrid cloud deployment scenarios and analyze its effectiveness in addressing the identified research challenges. The evaluation focuses on the feasibility of the proposed operational process, the integration of continuous security, the automation of security assessment, and the overall capability of the system to improve security operations within hybrid cloud environments.

1.6 Contributions

The main contributions of this research are summarized as follows:

- A SysSecOps operational process for hybrid cloud environments is proposed, integrating system administration, continuous security, and software delivery into a unified workflow. The proposed process addresses the operational challenges of heterogeneous infrastructure management, continuous security integration, and security automation identified in current hybrid cloud environments.

- A unified security assessment mechanism is developed by integrating multiple security analysis tools into a common security evaluation process. The proposed mechanism aggregates heterogeneous security findings and supports risk evaluation to improve security analysis and vulnerability prioritization. Its effectiveness is validated using

a security dataset containing both vulnerable and secure software samples.

- A prototype SysSecOps system is designed and implemented to realize the proposed operational process within a hybrid cloud environment. The prototype integrates infrastructure automation, continuous security assessment, and automated deployment workflows. Furthermore, representative experimental scenarios are conducted to evaluate the feasibility and effectiveness of the proposed approach in improving continuous security integration and operational security management.

1.7 Thesis organization

The remainder of this thesis is organized as follows.

Chapter 2 reviews the literature related to this research. It presents the fundamental concepts of cloud computing and hybrid cloud environments, followed by an overview of DevOps, DevSecOps, and SysSecOps. Existing security assessment approaches, risk evaluation methods, and current research on secure software delivery are also reviewed to establish the foundation and identify the limitations addressed by this research.

Chapter 3 presents the proposed SysSecOps process for hybrid cloud environments. It describes the overall system architecture, the operational workflow, the unified security assessment mechanism, and the proposed risk evaluation process. This chapter also explains how continuous security is integrated throughout the software operational lifecycle.

Chapter 4 describes the implementation of the proposed SysSecOps system and evaluates its effectiveness. The implementation environment, prototype system, experimental scenarios, and evaluation methodology are presented, followed by the experimental results and performance analysis.

Chapter 5 discusses the findings obtained from the experimental eval-

uation. The proposed approach is analyzed with respect to the research objectives and existing studies, while its limitations and potential improvements are also discussed.

Finally, Chapter 6 concludes the thesis by summarizing the research, highlighting its main contributions, and suggesting directions for future work.

Chapter 2

Related work

2.1 Cloud computing

The most widely accepted definition of cloud computing was proposed by the National Institute of Standards and Technology (NIST). Mell and Grance define cloud computing as "a model for enabling ubiquitous, convenient, on-demand network access to a shared pool of configurable computing resources (e.g., networks, servers, storage, applications, and services) that can be rapidly provisioned and released with minimal management effort or service provider interaction" [4]. This definition has become the de facto standard adopted by both academia and industry.

According to NIST, cloud computing is characterized by five essential characteristics: on-demand self-service, broad network access, resource pooling, rapid elasticity, and measured service [4]. These characteristics enable organizations to dynamically allocate computing resources according to workload demands while improving scalability and resource utilization. In addition, NIST categorizes cloud deployment into four models: private cloud, public cloud, community cloud, and hybrid cloud. Among these deployment models, hybrid cloud has gained increasing attention because it combines the flexibility and scalability of public cloud services with the control and security offered by private cloud infrastructures.

The widespread adoption of cloud computing has fundamentally trans-

formed modern software development and IT operations. Organizations increasingly deploy native cloud applications using automated development pipelines and distributed infrastructures, making security an essential consideration throughout the software operational lifecycle. As a result, cloud computing serves as the technological foundation for the hybrid cloud environments and SysSecOps process investigated in this research.

2.2 Hybrid cloud

Hybrid cloud is one of the four cloud deployment models defined by the National Institute of Standards and Technology (NIST). According to Mell and Grance, a hybrid cloud is "a composition of two or more distinct cloud infrastructures (private, community, or public) that remain unique entities but are bound together by standardized or proprietary technology that enables data and application portability" [4]. Unlike public or private cloud deployments alone, hybrid cloud allows organizations to distribute workloads across multiple computing environments while maintaining interoperability between them. This deployment model enables organizations to leverage the scalability and elasticity of public cloud services while retaining sensitive applications and data within private cloud infrastructures to satisfy security, regulatory, or organizational requirements.

To facilitate interoperability among heterogeneous cloud infrastructures, the National Institute of Standards and Technology further proposed the Cloud Computing Reference Architecture (CCRA), which describes the major actors, functional components, and interactions within cloud computing systems [5]. The reference architecture provides a standardized view of how cloud providers, cloud consumers, cloud brokers, cloud auditors, and cloud carriers interact to support cloud services. Within hybrid cloud environments, these components cooperate to enable workload portability, resource orchestration, service management, and communication across multiple cloud platforms, forming the architectural foundation

for modern hybrid cloud deployments.

The flexibility offered by hybrid cloud has led to its widespread adoption across both industry and academia. By combining public and private cloud resources, organizations can optimize infrastructure utilization, improve service availability, and dynamically allocate workloads according to business requirements while maintaining control over critical assets. Recent industry reports indicate that hybrid cloud has become the dominant deployment strategy for enterprise organizations, driven by the need to balance scalability, operational efficiency, regulatory compliance, and data governance [7], [8]. As cloud infrastructures continue to expand, organizations increasingly rely on hybrid cloud environments to support cloud-native applications, distributed services, and enterprise-scale software delivery.

Despite these advantages, hybrid cloud environments introduce significant operational complexity. Applications, services, and infrastructure components are distributed across heterogeneous platforms with different management interfaces, security policies, and deployment mechanisms. Maintaining consistent configurations, enforcing uniform security policies, and coordinating operational activities across multiple cloud environments become increasingly challenging as infrastructure complexity grows [5], [8]. These characteristics have motivated the adoption of operational methodologies that integrate infrastructure management, software delivery, and continuous security throughout the software operational lifecycle. Among these methodologies, DevOps and its security-oriented evolution, DevSecOps, have become prominent approaches for improving automation and collaboration in cloud environments.

2.3 DevOps and DevSecOps

The increasing adoption of cloud computing has significantly transformed the software development lifecycle, requiring organizations to de-

liver applications more rapidly while maintaining reliability and scalability. To address these demands, DevOps emerged as a software engineering methodology that promotes close collaboration between software development and IT operations through automation, continuous integration, continuous delivery, and continuous monitoring. Rather than treating development and operations as independent activities, DevOps integrates them into a continuous workflow that accelerates software delivery while improving consistent deployment and operational efficiency [13],[14].

The cloud computing paradigm has further accelerated the adoption of DevOps. Cloud platforms provide on-demand infrastructure, infrastructure-as-code (IaC), container orchestration, and automated deployment services that naturally complement DevOps practices. These capabilities enable organizations to provision infrastructure rapidly, automate software deployment, and continuously monitor running applications across distributed environments. Consequently, DevOps has become one of the predominant software delivery methodologies for cloud-native and hybrid cloud applications [13], [5].

Despite its advantages, DevOps primarily emphasizes development velocity, automation, and operational efficiency, while security activities have traditionally remained separate from the continuous development pipeline. Security assessments are frequently performed after software implementation or before deployment, increasing remediation costs and delaying software releases. As organizations increasingly deploy applications across cloud and hybrid cloud infrastructures, this separation becomes more problematic because security policies, infrastructure configurations, software dependencies, and cloud resources continuously evolve throughout the software operational lifecycle [11], [10].

To address these limitations, DevSecOps extends DevOps by integrating security practices into every stage of the software development and operational lifecycle. Rather than treating security as a separate verification phase, DevSecOps advocates continuous security through automated

security testing, vulnerability assessment, compliance verification, infrastructure security, and continuous monitoring. The NIST Secure Software Development Framework (SSDF) similarly recommends incorporating security practices throughout software development to reduce vulnerabilities and improve software integrity [9]. Recent literature reviews further identify continuous security integration, automation, and security orchestration as the defining characteristics of DevSecOps while highlighting that these remain active areas of research, particularly for complex cloud environments [11], [12].

Although DevSecOps significantly strengthens software security, existing research primarily concentrates on integrating security into software development workflows. Comparatively less attention has been devoted to systematically coordinating software development, infrastructure management, operational activities, and continuous security within heterogeneous hybrid cloud environments. This observation motivates the SysSecOps operational process proposed in this research, which extends existing DevSecOps practices by integrating system administration and infrastructure operations into a unified operational workflow for hybrid cloud environments.

2.4 Security Assessment and Risk Analysis

Security assessment plays a fundamental role in secure software development by identifying vulnerabilities throughout the software lifecycle before they can be exploited in production environments. Rather than treating security as a separate activity performed after software implementation, modern secure software development practices advocate continuous security assessment integrated into development, deployment, and operational processes. The NIST Secure Software Development Framework (SSDF) recommends incorporating automated security verification throughout the software development lifecycle to reduce software vulnerabilities and im-

prove software integrity [9]. Similarly, the OWASP Software Assurance Maturity Model (SAMM) identifies continuous verification, security testing, and defect management as essential practices for establishing mature software security programs [15].

A wide range of security assessment techniques has been proposed to identify vulnerabilities from different perspectives. Static Application Security Testing (SAST) analyzes source code without execution to detect programming flaws, insecure coding patterns, and potential vulnerabilities during software development. In contrast, Software Composition Analysis (SCA) focuses on identifying vulnerabilities introduced through third-party libraries and software dependencies by comparing dependency information against publicly disclosed vulnerability databases. Additional techniques, including infrastructure and configuration analysis, evaluate deployment environments and cloud infrastructure to identify security misconfigurations that may expose software systems to unnecessary risks. Because each technique targets different aspects of software security, they are generally considered complementary rather than interchangeable [9], [15].

As software systems become increasingly complex, organizations commonly employ multiple security assessment tools throughout the software lifecycle. Each tool produces findings using its own severity classification, confidence level, and reporting format, interpreting assessment results increasingly challenging. Consequently, modern vulnerability management extends beyond vulnerability detection to include vulnerability prioritization, enabling organizations to identify which security findings should be addressed first according to their potential impact and exploitation risk. NIST emphasizes that effective vulnerability management requires continuous identification, assessment, prioritization, remediation, and verification of security vulnerabilities throughout system operations rather than treating vulnerability scanning as an isolated activity [16].

Risk evaluation therefore has become an essential component of modern security assessment. The Common Vulnerability Scoring System (CVSS),

maintained by the Forum of Incident Response and Security Teams (FIRST), provides a standardized approach for measuring the severity of software vulnerabilities based on exploitability, impact, and environmental factors [17]. Although CVSS has become the de facto standard for vulnerability severity assessment, it primarily evaluates individual vulnerabilities rather than supporting organizational remediation decisions. To address this limitation, the Stakeholder-Specific Vulnerability Categorization (SSVC) framework incorporates additional decision factors, including exploitation status, mission impact, and operational context, to assist organizations in prioritizing vulnerability remediation according to their specific environments [18].

Recent research has increasingly explored the application of machine learning techniques to vulnerability assessment and security risk prediction. Unlike traditional rule-based scoring systems, machine learning models are capable of learning relationships among multiple security features and predicting vulnerability severity or exploitation likelihood from historical data. Various supervised learning algorithms have been investigated for this purpose, including Logistic Regression, Decision Trees, Random Forests, Support Vector Machines, and Gradient Boosting models. These approaches have been applied to vulnerability prioritization, exploit prediction, and software defect prediction, demonstrating that machine learning can improve the consistency and scalability of security risk evaluation by combining heterogeneous security indicators into a unified predictive model [19], [20], [21].

In practice, organizations often combine multiple security assessment techniques to obtain a more comprehensive understanding of software security. Static analysis, dependency analysis, infrastructure assessment, and compliance verification provide complementary perspectives on software risk, motivating the integration of heterogeneous security findings into unified security assessment workflows. Numerous tools have been developed to support these techniques, including Bandit for Python static anal-

ysis, Semgrep for pattern-based static analysis across multiple programming languages, and OWASP Dependency-Check for software dependency vulnerability analysis. While these tools improve vulnerability detection individually, effectively integrating their findings into a unified security evaluation process remains an important challenge for secure software operations. This challenge motivates the unified security assessment mechanism proposed in this research.

2.5 AI-Assisted Code Generation

Recent advances in generative artificial intelligence have significantly influenced modern software development practices. The introduction of the Transformer architecture established the foundation for large-scale language modeling by replacing recurrent computation with an attention mechanism that captures long-range dependencies more effectively [22]. Building upon this architecture, large language models (LLMs) trained on massive text corpora demonstrated the ability to perform a wide range of language tasks with minimal task-specific supervision [23]. When trained on large collections of publicly available source code, these models further acquired the capability to generate syntactically correct and functionally relevant program code from natural language descriptions, enabling a new paradigm of AI-assisted software development [24].

Large language models trained on code have since been integrated into practical software development tools, allowing developers to generate code, complete functions, and translate natural language requirements into executable programs. These capabilities extend naturally to infrastructure-as-code (IaC), where cloud infrastructure is defined declaratively through configuration templates or programmatic frameworks. By generating IaC artifacts directly from natural language prompts, AI-assisted code generation can substantially accelerate infrastructure provisioning and reduce the manual effort required to author complex cloud configurations. This capa-

bility is particularly valuable in hybrid cloud environments, where infrastructure must be provisioned consistently across heterogeneous platforms with different deployment mechanisms.

Despite these advantages, AI-assisted code generation introduces important security concerns. Because large language models learn statistical patterns from large-scale training data that may contain insecure or outdated coding practices, the generated code does not inherently guarantee security, correctness, or compliance with organizational policies. Empirical studies have shown that a significant proportion of code produced by AI programming assistants contains security vulnerabilities. Pearce et al. evaluated code generated by GitHub Copilot and found that a substantial fraction of the generated programs exhibited security weaknesses corresponding to well-known vulnerability categories [25]. Similarly, controlled user studies have observed that developers assisted by large language models tend to produce less secure code while simultaneously expressing greater confidence in its correctness, suggesting that AI assistance may inadvertently amplify security risks if left unchecked [26].

These concerns are particularly relevant in the context of infrastructure-as-code, where insecure configurations can directly expose cloud environments to attack. AI-generated IaC may introduce misconfigurations such as overly permissive access policies, unencrypted storage resources, or publicly exposed network endpoints, which can propagate throughout the infrastructure once deployed. In hybrid cloud environments, the impact of such misconfigurations is further amplified because infrastructure is distributed across multiple platforms with differing security controls. Consequently, the increased development velocity provided by AI-assisted code generation must be balanced with automated security verification to prevent insecure infrastructure from being deployed.

These observations motivate the integration of AI-assisted code generation with an automated security assessment mechanism within the operational workflow proposed in this research. Rather than treating AI-

generated infrastructure code as trustworthy by default, the generated artifacts are systematically evaluated through security scanning and risk assessment before deployment. Furthermore, the security findings identified during evaluation can be provided back to the generation process as feedback, enabling iterative refinement of the generated code. By coupling AI-assisted code generation with continuous security verification, the proposed approach aims to leverage the productivity benefits of generative AI while mitigating the security risks associated with automatically generated infrastructure-as-code.

Chapter 3

Proposed System

This chapter presents the proposed SysSecOps system for a hybrid cloud environment. It first describes the system architecture (Section 3.1), then the methodology of the proposed process in reproducible detail (Section 3.2), the risk-scoring engine that turns heterogeneous tool output into a single decision (Section 3.3), and finally how the methodology is realised in software (Section 3.4).

3.1 System Architecture

This section presents the architecture of the proposed SysSecOps system for a hybrid cloud environment. The design places a quantitative, multi-scanner *security gate* between the synthesis and the deployment of Infrastructure-as-Code (IaC), so that no change is applied to either the public-cloud or the on-premises side until it has been statically analysed, costed, checked against live compliance state, and assigned a single risk score. The same engine governs both sides of the hybrid environment, which gives the process a uniform and reproducible security behaviour.

3.1.1 Design goals

The architecture is driven by the research objectives stated in Chapter 1 and refined into the following concrete design goals:

- **Gate before deploy.** Security evaluation must occur on the *synthesized* artifact (the CloudFormation template or the resolved Ansible play), not merely on the higher-level source, because the synthesized artifact is the last representation before any resource is provisioned.
- **One engine for the whole hybrid.** The AWS (public) side and the on-premises (private) side must be scored with identical severity weights, deduplication rules, and decision thresholds, so that decisions are comparable across the trust boundary.
- **Graceful degradation.** A missing scanner binary or absent cloud credentials must never crash the pipeline; the affected component degrades to a well-defined "skipped" status while the gate still produces a decision.
- **Auditability.** Every decision (*pass*, *review*, *reject*) is persisted as an immutable record together with the acting identity, enabling post-hoc review.
- **Feedback-driven remediation.** When code is generated, gate findings are injected back into the generation prompt so that unsafe code is automatically regenerated rather than merely blocked.
- **Observability.** Every deployed target is continuously monitored and every gate decision is published as telemetry for dashboards and automated remediation.

3.1.2 Three-zone reference architecture

The system is organised into three cooperating zones, summarised in Table 3.1. Zone 1 produces IaC; Zone 2 evaluates and gates it; Zone 3 provisions the hybrid infrastructure and continuously observes it. The risk-scoring engine sits at the boundary of Zone 1 and Zone 2: it consumes scanner output and emits the decision that governs whether code advances to deployment or returns to Zone 1 for regeneration.

Table 3.1: The three zones of the proposed architecture.

Zone	**Name**	**Responsibility**
Zone 1	AI + IaC local development	Generate IaC from a natural-language prompt; local validation (`cdk synth`, `cdk diff`).
Zone 2	IaC security gate	Multi-scanner analysis, cross-source deduplication, risk scoring, and the deploy decision.
Zone 3	Hybrid infrastructure and operations monitoring	Provision AWS and on-premises resources; publish telemetry; run the drift/remediation operational loop.

Figure 3.1 shows the three zones and the principal flow of control between them. The solid arrows denote the forward path of a change from prompt to deployed and monitored infrastructure; the dashed arrow denotes the remediation feedback that returns rejected generated code to Zone 1.

The three zones are intentionally loosely coupled. Zone 1 knows nothing about the internal scoring rules of Zone 2 — it only consumes the machine-readable findings that Zone 2 emits. Zone 3 knows nothing about how a decision was reached — it only consumes the decision event. This decoupling is what allows the generation backend, the set of scanners, and the monitoring stack to evolve independently, and it is the reason the same

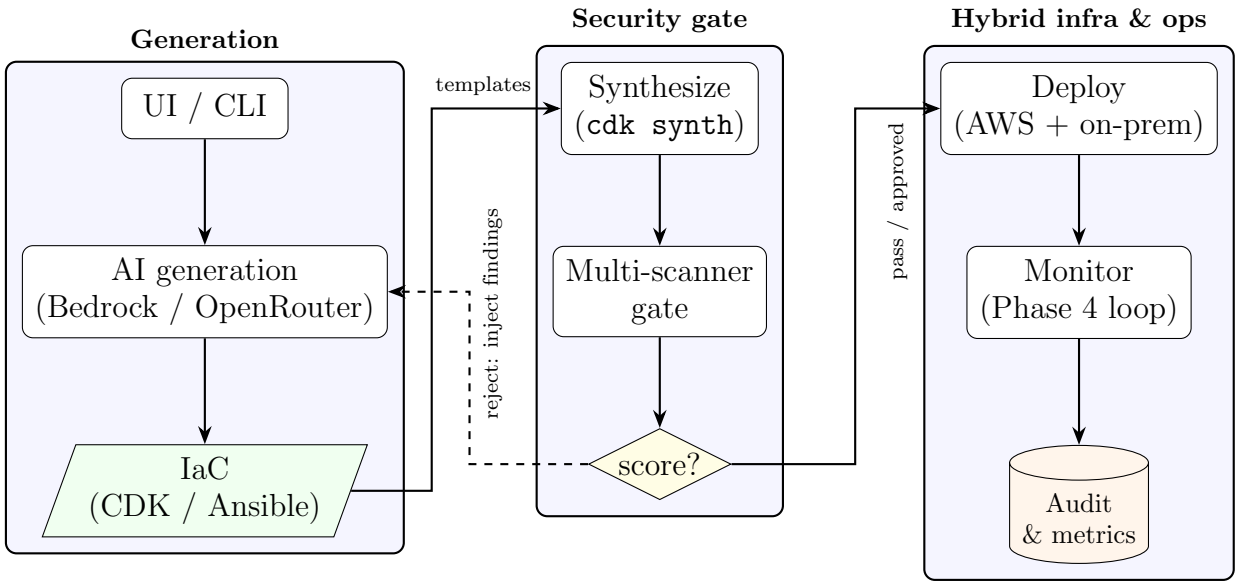

Figure 3.1: High-level three-zone architecture and end-to-end flow. Solid arrows are the forward path; the dashed arrow is the regeneration feedback loop.

gate can serve both the public and the private side without modification. The remainder of this subsection describes each zone, and the tools it uses, in turn.

Zone 1 — AI-assisted generation and local validation

Zone 1 is where Infrastructure-as-Code originates. It exposes two entry surfaces — a command-line interface and a Streamlit operator console — and turns a natural-language prompt into synthesizable IaC. Code generation is delegated to a large-language-model backend behind a provider abstraction, so that the concrete model is a configuration choice rather than a code dependency; the reference implementation supports Amazon Bedrock and OpenRouter. The generated artifact is an AWS Cloud Development Kit (CDK) Python application for the public side and/or a set of Ansible playbooks for the private side.

Before any artifact leaves Zone 1, it is validated locally so that only

well-formed input reaches the gate. For the public side, `cdk synth` synthesizes the CloudFormation template and `cdk diff` summarises the change against the currently deployed stack; for the private side, `ansible-playbook --syntax-check` (and an optional `--check` dry run) validates the plays. Zone 1 also closes the *generate* → *synth* → *gate* → *regenerate* feedback loop: when the gate rejects generated code, the findings are serialised back into the next prompt so that the model remediates its own output within a bounded attempt budget. Table 3.2 lists the tools used in this zone.

Table 3.2: Tools used in Zone 1 (generation and local validation).

Tool	**Role in Zone 1**
Amazon Bedrock / OpenRouter	Interchangeable LLM backends that generate CDK or Ansible code from a prompt.
AWS CDK (Python)	Expresses public-cloud infrastructure as code and synthesizes it to CloudFormation.
`cdk synth` / `cdk diff`	Produces the CloudFormation template and the change summary used for local validation.
Ansible	Expresses on-premises configuration as playbooks.
`ansible-playbook --syntax-check`	Validates playbook syntax (and, optionally, a dry-run `--check`) before gating.

Zone 2 — the multi-scanner security gate

Zone 2 is the core contribution of the architecture: a quantitative security gate that consumes the synthesized artifact and emits a single decision. It runs a set of independent, best-of-breed scanners, normalises their heterogeneous outputs into a common finding schema, deduplicates findings that several tools report for the same resource/template/category, aggregates the surviving findings into a numeric risk score, and maps that score to *pass*, *review*, or *reject*. Each scanner is wrapped in an adapter that never raises: a missing binary or an absent credential degrades to

a well-defined *skipped* status, and the gate still produces a decision (the graceful-degradation contract).

The tools are deliberately complementary and cover four dimensions of risk — *misconfiguration*, *cost*, *live compliance*, and *code-level insecurity*. Static policy-as-code analysis (Checkov) and CloudFormation validation (`cfn-lint`) inspect the public-side template; `ansible-lint` inspects the private-side plays; a built-in regular-expression scanner detects hard-coded secrets; Infracost supplies the estimated monthly cost that drives the cost dimension; AWS Config supplies the count of live non-compliant rules; and a machine-learning code-risk model (trained on Bandit and Semgrep features, see Section 3.3) contributes a probability of insecurity. Table 3.3 lists these tools and the signal each contributes.

Zone 3 — hybrid infrastructure and operations

Zone 3 provisions the hybrid infrastructure that survived the gate and continuously observes it. The public side is provisioned on Amazon Web Services (AWS) by deploying the CDK-synthesized CloudFormation; the private side consists of on-premises Linux nodes configured by Ansible. After a successful deployment, a three-layer monitoring mesh is attached to the public resources and the on-premises nodes are registered so that both sides are watched by a single operational loop.

On the public side, AWS Systems Manager (SSM) provides Run Command, State Manager, Patch Manager, Inventory, and Parameter Store; Amazon EventBridge carries compliance and stack-rollback events; Amazon CloudWatch stores the gate metrics, alarms, dashboards, and Application Insights views; AWS CloudTrail records the API audit trail; an AWS Lambda function implements the event-driven operational loop (drift reaction and pipeline re-trigger); and Amazon SNS delivers operator notifications. On the private side, the SSM Agent together with an SSM *hybrid activation* registers each on-premises node as a managed instance (`mi-*`), so the same Run Command, drift-detection, patch, and compliance ma-

Table 3.3: Tools used in Zone 2 (the security gate) and the signal each contributes.

Tool	**Signal**	**Dimension**
AWS CDK (`cdk synth`)	Emits the CloudFormation template that is the analysed artifact.	pre-validation
`cfn-lint`	CloudFormation validity and best-practice findings.	misconfig.
Checkov	Policy-as-code misconfiguration findings on the templates.	misconfig.
`ansible-lint`	Best-practice and security findings on the on-premises plays.	misconfig.
Regex secret scanner (built-in)	Hard-coded credentials and secrets.	misconfig.
Infracost	Estimated monthly cost / cost delta of the change.	cost
AWS Config	Count of non-compliant live-compliance rules.	compliance
ML code-risk model (Bandit + Semgrep → logistic regression)	Probability that the generated code is insecure, converted to points.	code risk

chinery used for EC2 also covers the on-premises fleet without a separate agent stack. Table 3.4 summarises the tooling.

3.1.3 End-to-end data flow

A single request flows through the system as an evolving chain of artifacts:

1. A natural-language prompt (or an existing, "bring-your-own" project) enters the system through the command-line interface or the operator

Table 3.4: Tools used in Zone 3 (hybrid infrastructure and operations).

Side	**Tool / service**	**Role**
Public	AWS CDK / CloudFormation	Provisions and updates the public-cloud resources.
Public	EC2, S3, RDS, VPC	The workload substrate that the CDK app declares.
Public	AWS Systems Manager	Run Command, State Manager, Patch, Inventory, and Parameter Store (the gate-decision store).
Public	Amazon EventBridge	Event bus for compliance-change and stack-rollback events.
Public	Amazon CloudWatch	Gate metrics, alarms, dashboards, and Application Insights.
Public	AWS CloudTrail	Multi-region API audit trail feeding CloudWatch Logs.
Public	AWS Lambda	The ops-loop function: drift reaction and pipeline re-trigger.
Public	Amazon SNS	Operator notifications for alarms and rollbacks.
Private	Ansible	Configuration management of the on-premises Linux nodes.
Private	SSM Agent + hybrid activation	Registers each node as an `mi-*` managed instance so it joins the same operational loop.
Shared	Secure connectivity link	Joins the two sides through one of several options (see Section 3.1.4).

console.

2. Zone 1 produces IaC: a synthesizable AWS Cloud Development Kit (CDK) Python application and/or a set of Ansible playbooks.

3. The IaC is synthesized: `cdk synth` emits CloudFormation templates, while `ansible-playbook --syntax-check` validates the playbooks.

4. Zone 2 evaluates the synthesized artifact with several independent scanners, normalises their outputs into a common finding schema,

deduplicates across sources, and computes a risk score.

5. The decision function maps the score to *pass*, *review*, or *reject*. The full gate report is persisted, and an approval or rejection record is written.

6. On *pass* (or an approved *review*), the change is deployed; on *reject*, generated code is optionally regenerated with the findings injected back into the prompt.

7. Zone 3 records the outcome, publishes metrics and events, and attaches post-deployment security monitoring.

Figure 3.2 traces the same request as a chain of artifacts, making explicit how each stage transforms its input into a more concrete representation. The important property is that the security decision is taken on the *synthesized template*, not on the prompt or the high-level source: this is the last artifact before provisioning and therefore the earliest point at which the exact resources, their properties, and their relationships are fully known.

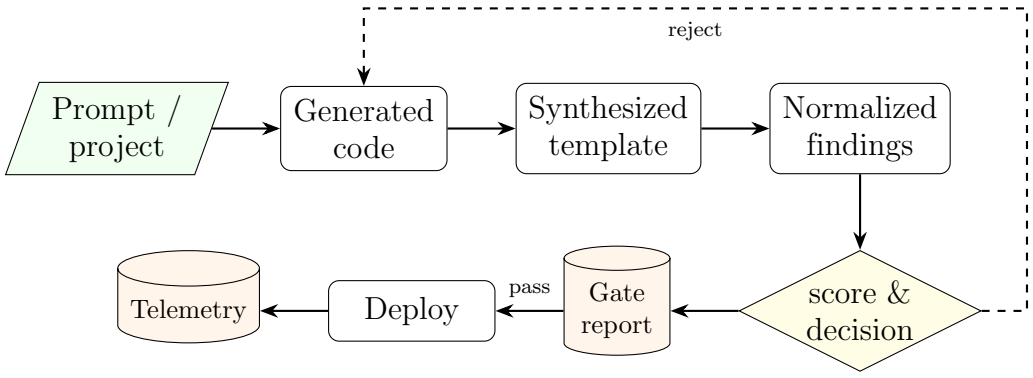

Figure 3.2: Artifact lifecycle of a single request. The decision is computed on the synthesized template; a rejection feeds findings back into generation.

3.1.4 Hybrid deployment topology

The public side is provisioned on Amazon Web Services (AWS) through CDK-synthesized CloudFormation. The private side consists of on-premises Linux nodes managed by Ansible. The two sides are joined by an encrypted link, and the architecture deliberately treats *how* that link is established as a pluggable concern: Section 3.1.4 surveys the available connectivity options. The on-premises nodes are registered as AWS Systems Manager (SSM) managed instances so that their compliance state is visible to the same operational loop that watches the cloud side. No public inbound administrative access is required: the control plane traverses the encrypted link, which keeps the hybrid trust boundary small and observable. Figure 3.3 depicts the topology with the connectivity method shown as an abstract secure channel.

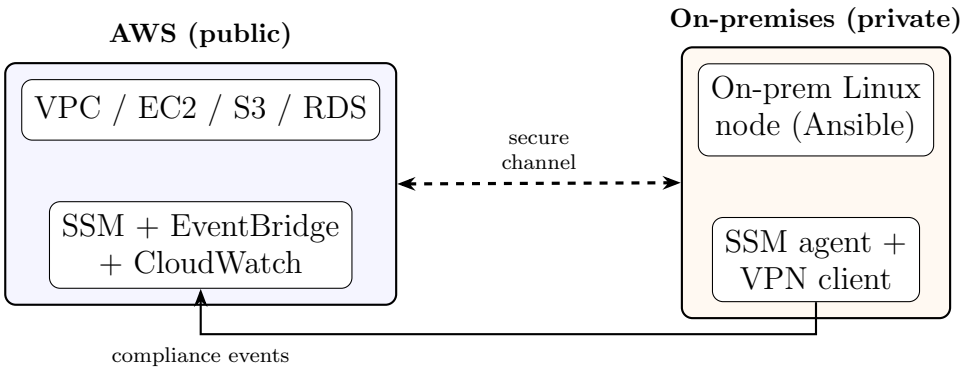

Figure 3.3: Hybrid deployment topology: AWS resources and on-premises nodes joined by a secure channel (any of the options in Table 3.5), with on-premises nodes registered as SSM managed instances.

Connectivity options between the public and private sides

Joining the public and private sides is a design choice with several viable options rather than a single fixed mechanism. The architecture treats connectivity as a pluggable concern: the gate, the SSM registration, and the operational loop are indifferent to *how* the link is established,

Table 3.5: Options for connecting the public (AWS) and private (on-premises) sides.

Method	**Mechanism**	**Trade-offs**
Mesh overlay VPN (Tailscale / WireGuard)	Userspace WireGuard tunnels with automatic NAT traversal and identity-based access control lists; each node runs a lightweight agent.	Fast to set up, no public inbound ports, per-node identity; depends on a software agent and a coordination service.
AWS Site-to-Site VPN (IPsec)	Managed IPsec tunnels between an on-premises customer gateway device and an AWS virtual private gateway or transit gateway.	Standards-based and fully encrypted over the public Internet; needs a compatible gateway device and is bandwidth-limited per tunnel.
AWS Direct Connect	A dedicated private physical circuit from the on-premises data centre into AWS, bypassing the public Internet.	Consistent low latency and high, stable bandwidth; higher cost and provisioning lead time, and usually paired with a VPN for encryption.
AWS Transit Gateway / VPN CloudHub	A managed routing hub that interconnects many VPCs and multiple on-premises sites in a hub-and-spoke topology.	Centralises and scales routing across sites; adds a managed routing tier and its associated cost.
SSM hybrid activations over public endpoints	No network tunnel: the SSM Agent reaches the regional Systems Manager endpoints directly over TLS 443.	Simplest option for management-plane reach only; provides no private data-plane connectivity between workloads.

as long as the on-premises nodes can reach the regional AWS endpoints and the two sides can exchange traffic securely. Table 3.5 compares the

practical options, ordered from the lightest-weight software overlay to a dedicated physical link, so that a deployment can select the method that matches its latency, bandwidth, cost, and compliance requirements.

These options are not mutually exclusive: a lightweight mesh overlay can carry workload and control traffic while an SSM hybrid activation provides the operational loop with management-plane reach, and the heavier options (IPsec Site-to-Site VPN, Direct Connect, or a Transit Gateway hub) can be substituted where standards-based encryption, dedicated bandwidth, or multi-site routing is required. Crucially, the choice of connectivity is orthogonal to the rest of the pipeline: none of these methods changes the gate, the risk scoring, or the deployment-governance behaviour, which is what allows the architecture to remain portable across different hybrid networking substrates.

3.1.5 Component map and interfaces

Table 3.6 lists the top-level software modules and their responsibilities. The modules communicate through stable contracts rather than shared internal state, which keeps the generation, evaluation, orchestration, and monitoring concerns decoupled.

3.1.6 Key data contracts

Three contracts hold the system together and are deliberately kept stable so that independent consumers (the CLI, the console, the dashboards) do not break:

- **Gate report.** A JSON document containing the run identifier, timestamp, numeric `score`, `decision`, human-readable `message`, a per-component `components` breakdown, the deduplicated `findings`, and a `scanner_status` map that records whether each scanner ran, was skipped, or was unavailable.

Table 3.6: Top-level modules and their responsibilities.

Module	Responsibility
`generation/`	Zone 1 generation: provider-abstracted code generation (Bedrock / OpenRouter) and the CDK regeneration loop.
`security_gate/`	Zone 2 gate: template and playbook analysis, scanner adapters, deduplication, risk scoring, and report assembly.
`execution/`	Subprocess execution backbone with a stable result contract.
`pipeline/`	Orchestration and governance: synth–gate–diff–deploy, the approval audit trail, credentials, and the Phase 4 emitters.
`monitoring/`	CloudTrail, CloudWatch metrics and dashboards, the operational-loop Lambda, and on-premises SSM registration.
`scripts/`	Command-line entry points for the CDK-only and the hybrid flows.
`ui/`	Streamlit operator console and the multi-project registry.
`risk_scoring/`	Training and evaluation code for the machine-learning code-risk model consumed by the gate.

- **Execution result.** Every external command returns exactly `{return_code, output}`, with standard output and standard error merged to preserve event ordering for the console and the logs.

- **Gate-decision event.** A compact event (target name, score, decision, timestamp) is persisted to SSM Parameter Store and published to Amazon EventBridge, keyed by the target name (a stack name for CDK, or `ansible-<node>` for the on-premises side) so that the two sides never collide.

3.1.7 Security and trust model

Credentials are never passed as command-line arguments; they are resolved through the standard provider chains and injected into subprocesses through the environment. AWS access follows the default credential chain with a single-sign-on fallback, and generation-provider keys are read from the environment. Deployed workloads follow the principle of least privilege, and each privileged action is recorded in an append-only audit trail. The uniform graceful-degradation contract guarantees that the absence of a tool or a credential weakens visibility but never silently disables the gate.

3.2 Proposed Methodology

This section describes the methodology of the proposed process in enough detail that the pipeline could be reimplemented from the description. The approach reuses an established security-automation baseline and extends it with three additions tailored to hybrid cloud: (i) an AI-assisted generation stage with a closed remediation loop, (ii) a unified risk-scoring engine that aggregates heterogeneous tool findings into a single decision, and (iii) a uniform application of the same gate to both the public and the private side of the environment.

3.2.1 Approach overview

Each hybrid-cloud challenge identified in Chapter 1 is mapped to a concrete module that addresses it. Table 3.7 makes this mapping explicit and provides the traceability from problem to solution that motivates the remainder of the section.

Table 3.7: Mapping from hybrid-cloud challenges to solution modules.

Challenge	**Module / mechanism**
Weak security linkage between public and private cloud	One gate and one risk engine applied to both the CDK and the Ansible side.
Absence of a clear, repeatable process	The synth–gate–decide–deploy pipeline with fixed thresholds and audit records.
Fragmented tooling with inconsistent output	Scanner adapters that normalise findings into a common schema, followed by cross-source deduplication.
Misconfiguration risk amplified by fast AI generation	The generate–gate–regenerate feedback loop that injects findings back into the prompt.
Context-free prioritisation	A risk score that combines severity with cost, live compliance, and a machine-learning code-risk signal.

3.2.2 AI-assisted generation and the regeneration loop

Zone 1 generates synthesizable CDK Python code from a natural-language prompt. Code generation is placed behind a provider abstraction so that two backends — a managed foundation-model service and a hosted inference API — are interchangeable behind a single provider flag and share an identical generation contract. Model defaults are centralised and can be overridden explicitly.

Generation alone does not guarantee secure output, so it is wrapped in a closed loop. The loop generates code, synthesizes it, gates it, and — if the gate rejects — rebuilds the prompt with the machine-readable findings injected before retrying, up to a bounded number of attempts. Each attempt is persisted for reproducibility under `logs/cdk_regen/<run_id>/attempt_<N>/` (the prompt snapshot, the generated code, the synthesis output, and the

gate report), and the successful attempt is promoted to a `passed/` folder. Algorithm 1 formalises the loop.

Algorithm 1 Generate–synthesize–gate–regenerate loop

Require: prompt q, maximum attempts N
Ensure: gated CDK application, or failure after N attempts

$findings \leftarrow \emptyset$
for $i \leftarrow 1$ to N **do**
  $code \leftarrow \textsc{Generate}(q, findings)$
  write *code* to the CDK application
  $t \leftarrow \textsc{Synth}(code)$ ▷ emit CloudFormation templates
  if SYNTH failed **then**
    $findings \leftarrow$ synthesis error
    continue
  $report \leftarrow \textsc{Gate}(t)$
  if $report.decision \neq reject$ **then**
    return *report* ▷ pass or review
  $findings \leftarrow report.findings$ ▷ injected into the next prompt
return failure

Figure 3.4 presents the same loop as a flowchart, emphasising the two exit conditions: an accepting decision (*pass* or *review*) leaves the loop with a gated application, whereas exhausting the attempt budget N leaves it with a failure that surfaces to the operator.

3.2.3 Pre-validation

Before any security analysis, each side is validated syntactically. On the CDK side, `cdk synth` must succeed (otherwise the loop treats the failure as feedback), and `cdk diff` previews the change. On the Ansible side, `ansible-playbook --syntax-check` validates the plays and `ansible-playbook --check --diff` performs a dry run. This staging (*validate* → *dry run* → *deploy*) is symmetric across the two sides and ensures the gate only ever scores artifacts that are at least well formed.

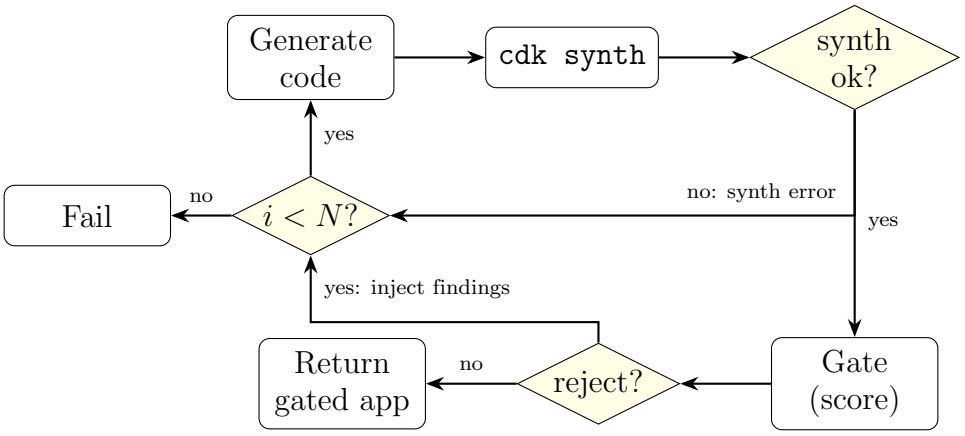

Figure 3.4: Regeneration loop as a flowchart. The loop exits on an accepting decision or when the attempt budget N is exhausted.

3.2.4 Security evaluation and cross-source deduplication

Zone 2 analyses the synthesized artifact with several independent scanners, each wrapped in an adapter that normalises its output into a common finding schema with the fields `severity`, `source`, `message`, `resource_id`, `template`, and `category`. Because several tools frequently report the same underlying issue (for example, an open administrative port may be flagged by both a heuristic check and a policy scanner), the findings are deduplicated by the key (`resource_id`, `template`, `category`). When duplicates collide, the highest severity is retained and the source labels are merged. Deduplication is essential: without it, redundant reports would inflate the score and distort the decision. Figure 3.5 shows the analysis pipeline: the synthesized artifact fans out to the parallel scanner adapters, whose normalized findings are merged, deduplicated, and passed to the risk-scoring engine.

3.2.5 Risk-scoring engine (overview)

The deduplicated findings, together with the cost, live-compliance, and machine-learning code-risk signals, are aggregated by the gate into a single

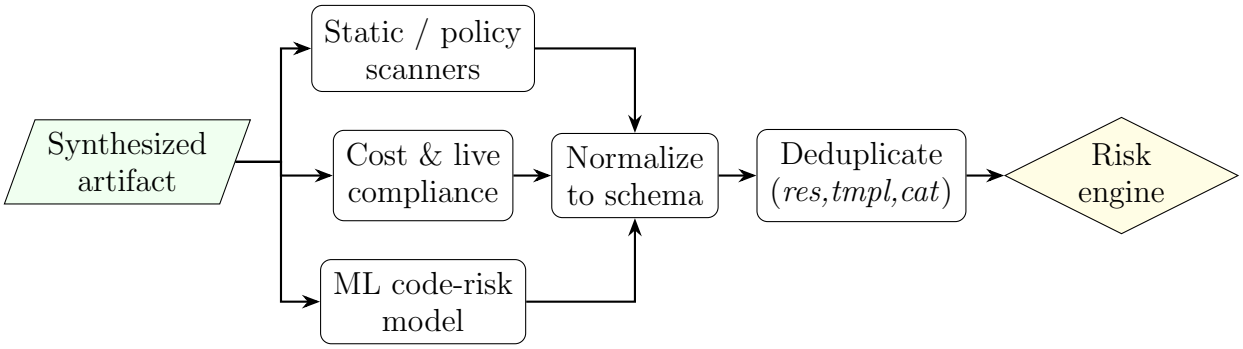

Figure 3.5: Security-evaluation pipeline: parallel scanners, normalization to a common schema, cross-source deduplication, and aggregation by the risk-scoring engine.

numeric score that is then mapped to one of three decisions — *pass*, *review*, or *reject*. Because this aggregation is the analytical heart of the proposed process, it is presented in full in Section 3.3, which develops the multi-factor formulation, the additive scoring implementation, the machine-learning code-risk model, the deduplication-driven explainability, and a worked example. Here it suffices to note that the score is an *additive* combination of weighted findings, banded cost, live compliance violations, and a bounded machine-learning contribution, and that the decision thresholds are fixed so that identical inputs always yield identical decisions on both sides of the hybrid environment.

3.2.6 Deployment governance

The decision governs deployment. A *pass* deploys automatically. A *review* requires an explicit approval, which is recorded together with the approver's cloud identity, the run identifier, the score, and the decision. A *reject* never deploys; when the code was generated, the regeneration loop is invoked, and otherwise a rejection record is written for the author to act on. Approval and rejection records are append-only, which yields the immutable audit trail required by the design goals.

3.2.7 Monitoring and the operational loop

After a successful deployment, Zone 3 attaches post-deployment monitoring and closes an operational loop. Each gate decision is persisted to SSM Parameter Store, published as an EventBridge event, and emitted as CloudWatch metrics and structured logs. An event-driven function reacts to compliance drift and rollback events, enabling automated remediation. Because the on-premises nodes are registered as managed instances, their compliance events flow through the same loop, so the entire hybrid environment is observed uniformly rather than as two disconnected halves.

3.3 Risk-Scoring Engine

The risk-scoring engine is the analytical core of the proposed process: it converts the heterogeneous output of several independent security tools into one number and one decision. This section motivates why a single, purpose-built score is needed (Section 3.3.1), states the multi-factor formulation (Section 3.3.2), describes the additive scoring actually implemented in the gate (Section 3.3.3), details the machine-learning code-risk model (Section 3.3.4), explains how deduplication yields explainability (Section 3.3.5), and closes with a worked example (Section 3.3.6).

3.3.1 Motivation and limitations of severity-only scoring

Industry practice prioritises vulnerabilities almost exclusively by technical severity, typically a Common Vulnerability Scoring System (CVSS) base score. For infrastructure changes in a hybrid cloud this is insufficient for three reasons:

- **Severity ignores blast radius and cost.** An open security-group rule on an internet-facing load balancer and the same rule on an iso-

lated subnet may share a severity label yet differ enormously in exposure. Likewise, a change that quietly provisions expensive resources carries an operational risk that no security-only score captures.

- **Single tools have blind spots.** A policy scanner, a template linter, a cost estimator, a live-compliance service, and a code-level model each see a different slice of the same change. Relying on any one of them yields systematic false negatives.
- **AI generation shifts the risk profile.** When code is produced quickly by a language model, the dominant failure mode is not a novel exploit but a plausible misconfiguration. A signal that estimates the *probability* that generated code is insecure is therefore as valuable as any individual rule.

The engine addresses these limitations by combining several orthogonal signals into one additive score with a fixed, auditable decision boundary, so that prioritisation reflects severity *and* exposure, cost, live compliance, and a learned code-risk estimate.

3.3.2 Multi-factor formulation

Conceptually, the risk of a change is a weighted combination of five factors:

- **Severity** (S): the technical seriousness of each finding.
- **Exposure** (X): how reachable the affected resource is (for example, public versus private).
- **Exploitability** (E): how readily a finding can be abused.
- **Confidence** (C): how certain the detecting tool is, which discounts low-confidence heuristics.

- **Cost** (K): the financial blast radius of the change.

A general multi-factor risk of a change with finding set F can be written as

$$R \;=\; \sum_{f \in F} \alpha\, S_f \cdot X_f \cdot E_f \cdot C_f \;+\; \beta\, K, \tag{3.1}$$

where α and β balance the finding-driven term against the cost term. This formulation is the design intent; the implemented engine (next subsection) is a deliberately simplified, *additive* realisation of it that keeps the score transparent and reproducible while folding exposure, exploitability, and confidence into the calibrated severity weights and the live-compliance term.

3.3.3 Implemented additive scoring

The gate implements Equation 3.1 as an additive sum of four components that are easy to explain and cheap to compute. Each deduplicated finding contributes points according to its severity (Table 3.8); a monthly cost increase contributes points in bands; each non-compliant live rule contributes a fixed number of points; and the machine-learning model contributes points proportional to its predicted probability of insecurity.

Table 3.8: Severity-to-points mapping used by the gate.

Severity	**Points**
Critical	20
High	10
Medium	5
Low	1

Formally, let F be the set of deduplicated findings, $\mathrm{sev}(f)$ the severity of finding f, and $w(\cdot)$ the mapping in Table 3.8. Let Δ be the estimated monthly cost increase in US dollars, $c(\Delta)$ the banded cost contribution,

v the number of non-compliant live-compliance rules, and $p \in [0, 1]$ the predicted probability of insecurity from the machine-learning model. The total score S is

$$S = \underbrace{\sum_{f \in F} w\big(\mathrm{sev}(f)\big)}_{\text{severity}} + \underbrace{c(\Delta)}_{\text{cost}} + \underbrace{5\,v}_{\text{compliance}} + \underbrace{\mathrm{round}\big(p \cdot P_{\mathrm{ml}}\big)}_{\text{ML code-risk}}, \tag{3.2}$$

where $P_{\mathrm{ml}} = 20$ is the maximum contribution of the machine-learning component at $p = 1$. The cost contribution $c(\Delta)$ is banded: a high band applies above a high monthly-cost threshold and a medium band above a medium threshold, with the thresholds and their points being configurable. Banding, rather than a continuous cost term, keeps the score stable against small estimation noise.

The decision function maps S to one of three outcomes using the thresholds in Table 3.9:

$$\mathrm{decision}(S) = \begin{cases} \textit{pass} & \text{if } S \leq 20, \\ \textit{review} & \text{if } 20 < S \leq 80, \\ \textit{reject} & \text{if } S > 80. \end{cases} \tag{3.3}$$

Table 3.9: Decision thresholds and their operational meaning.

Decision	**Score band**	**Action**
Pass	≤ 20	Auto-deploy; no human review required.
Review	21–80	Manual approval required before deployment.
Reject	> 80	Do not deploy; regenerate (if generating) or return to the author.

Figure 3.6 summarises the engine: the four component sub-scores are

summed and compared against the two thresholds to select the decision.

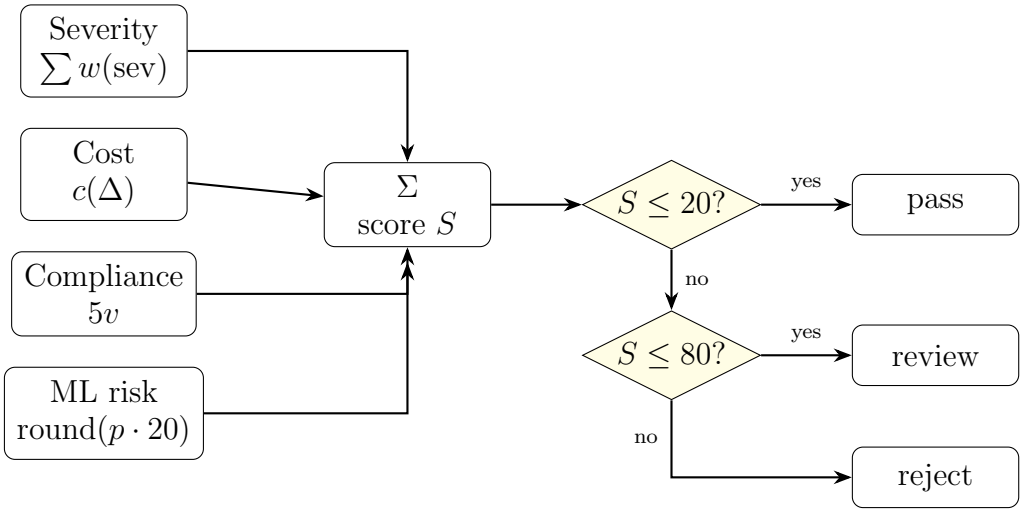

Figure 3.6: The additive risk-scoring engine: four orthogonal sub-scores are summed and the total is compared against two fixed thresholds to yield the decision.

3.3.4 Machine-learning code-risk model

The machine-learning component predicts whether generated Python code is likely to be insecure, supplying the probability p used in Equation 3.2. Unlike the rule scanners, which fire only on patterns encoded in their rule sets, this component learns a statistical association between static-analysis signals and ground-truth insecurity, so it can flag code whose overall signal profile resembles known-insecure samples. Its construction has three parts: a labelled dataset, a feature-extraction step shared between training and inference, and a logistic-regression classifier.

Dataset construction

The training corpus pairs insecure and secure Python files under a common schema. The insecure class is drawn from the SecurityEval dataset, a public benchmark of CWE-labelled insecure Python snippets, each of which is materialised to a temporary file and scanned. The secure

class is sampled from the source trees of well-maintained, widely-used open-source projects (for example `click`, `rich`, and `typer`), whose code is taken as a negative (secure) reference. To keep the two classes comparable and to remove degenerate inputs, a filtering rule admits a file only if it is not a package initialiser or a test, and contains between 20 and 700 non-blank, non-comment lines; overly short or generated-boilerplate files are therefore excluded. Each admitted file becomes one labelled sample — label 0 for secure, label 1 for insecure — yielding a balanced dataset of roughly 150 samples split almost evenly between the two classes.

A practical finding shaped the final corpus. An initial run showed that a large fraction of the SecurityEval insecure samples (on the order of half, spanning dozens of distinct CWEs) contain flaws — chiefly logic and mis-counting errors — that the two static analysers cannot detect, so those samples were statistically indistinguishable from secure code under the chosen features. Retaining them capped recall (an initial model reached only ≈ 0.73 accuracy with recall near 0.46). Removing these undetectable outliers, while keeping a small margin ($\approx 15\%$) of them so the model still sees hard negatives, produced a corpus on which the same features are informative. This is an explicit *scope statement* for the model: it estimates the risk of *detector-visible* insecurity, not of semantic flaws invisible to `bandit` and `semgrep`.

Feature extraction

Each file is scanned independently by two static analysers — `bandit` (a security-focused linter) and `semgrep` (a pattern-matching analyser run with its automatic rule set) — and their findings are reduced to an eleven-dimensional count vector (Table 3.10). Counts, rather than raw messages, keep the feature space small, tool-version-robust, and directly comparable across files.

Table 3.10: Per-file feature vector extracted for the code-risk model.

Feature group	**Source**	**Features**
Severity counts	`bandit`	`bandit_high`, `bandit_medium`, `bandit_low`
Confidence counts	`bandit`	`bandit_conf_high`, `bandit_conf_medium`, `bandit_conf_low`
Severity counts	`semgrep`	`semgrep_high`, `semgrep_medium`, `semgrep_low`
Totals	both	`total_bandit`, `total_semgrep`

Training procedure

The classifier is a logistic-regression model trained with the configuration in Table 3.11. The dataset is split 80/20 into train and test partitions with stratification on the label, so both partitions preserve the class balance. Features are standardised (zero mean, unit variance) with a scaler fitted on the training partition and re-applied unchanged at test and inference time. Because the two classes are close to but not exactly balanced, the loss uses balanced class weights so that neither class dominates the fit. The fitted model exposes, for each file, the probability of the positive (insecure) class via its sigmoid output.

At inference time the gate applies the model to every generated file and takes the worst (maximum) probability, so that a single risky file cannot be masked by many benign ones:

$$p \;=\; \max_{k \in \text{files}} \sigma\big(\mathbf{w}^{\top}\mathbf{x}_k + b\big)\,, \qquad \sigma(z) = \frac{1}{1+e^{-z}}, \tag{3.4}$$

where $\mathbf{x}_k$ is the per-file feature vector of Table 3.10, and $\mathbf{w}, b$ are the learned parameters. The decisive design constraint is *inference/training parity*: feature extraction at inference time uses the same two analysers, the

Table 3.11: Training configuration of the logistic-regression code-risk model.

Setting	**Value**
Algorithm	Logistic regression (regularised, `max_iter`= 1000)
Train/test split	80%/20%, stratified on the label
Feature scaling	Standardisation fitted on the training partition only
Class weighting	Balanced (inversely proportional to class frequency)
Output	P(insecure) per file via the sigmoid
Persisted artifacts	Model and scaler (serialised) for reuse by the gate

same severity and confidence grouping, and the same per-file scanning as during training, and it reuses the *persisted scaler*, so the inputs seen by the classifier match its training distribution. When the model artifacts or the analyser binaries are unavailable, the component degrades to a "skipped" status and contributes zero points, consistent with the graceful-degradation contract of the overall system.

The learned coefficients are also interpretable and act as a sanity check: the `semgrep` total, the medium-confidence `bandit` count, and the high-severity `semgrep` count carry the largest positive weights, matching the intuition that a higher volume of medium-to-high-confidence findings is the strongest indicator of insecurity. On the cleaned corpus the model reaches roughly 0.87 accuracy and 0.88 ROC-AUC; the full evaluation, including the comparison against baseline scorers, is reported in the results chapter.

3.3.5 Deduplication and explainability

Two properties make the score trustworthy. First, *deduplication* guarantees that no single underlying issue is counted twice: findings are keyed by (`resource_id`, `template`, `category`), and colliding findings are merged, keeping the highest severity and unioning the source labels. Without this

step, an issue reported by three tools would triple its severity contribution and could push an otherwise acceptable change over a threshold.

Second, *explainability* follows directly from the additive form of Equation 3.2: because the score is a sum of clearly labelled components, the gate report includes a per-component breakdown that attributes exactly how many points came from severity, cost, compliance, and the machine-learning model. An operator reviewing a *review* or *reject* decision can therefore see *why* the score landed where it did and which component to remediate, rather than being handed an opaque number.

3.3.6 Worked example

Consider a change whose deduplicated findings include one high-severity open administrative port and two medium-severity issues, whose estimated monthly cost increase falls in the medium band (contributing 10 points), which trips one non-compliant live rule, and whose generated code the machine-learning model scores at $p = 0$ (secure). Applying Equation 3.2:

$$\begin{aligned} S &= \underbrace{(10 + 5 + 5)}_{\text{severity}} + \underbrace{10}_{\text{cost}} + \underbrace{5 \times 1}_{\text{compliance}} + \underbrace{\text{round}(0 \times 20)}_{\text{ML}} \\ &= 20 + 10 + 5 + 0 \;=\; 35. \end{aligned}$$

Since $20 < 35 \leq 80$, Equation 3.3 yields *review*: the change is not auto-deployed but is instead routed to a human approver, whose approval (or rejection) is recorded in the audit trail. The per-component breakdown (severity = 20, cost = 10, compliance = 5, ML = 0) tells the approver that the open administrative port and the cost increase are the dominant contributors, directing remediation effort precisely.

3.4 Implementation

This section describes how the proposed methodology is realised in software. The system is implemented in Python and is exposed both as a set of command-line entry points and as a browser-based operator console. The implementation follows the module boundaries introduced in Section 3.1: generation, evaluation, execution, orchestration, monitoring, and the user interface.

3.4.1 Technology stack and repository layout

The public side is defined with the AWS Cloud Development Kit for Python and synthesized to CloudFormation; the private side is defined with Ansible playbooks. Static analysis relies on established open-source scanners, cost estimation on a cost-analysis tool, and live compliance on the cloud provider's configuration service. The operator console is built with Streamlit. Table 3.12 lists the directories that implement each concern.

Figure 3.7 shows how these directories depend on one another at runtime. The command-line entry points in `scripts/` sit at the top; they call into `pipeline/` for orchestration, which in turn drives generation (`generation/`), evaluation (`security_gate/`), and monitoring (`monitoring/`). Every directory that runs an external process funnels through the single execution helper in `execution/`, and the operator console in `ui/` reuses the same entry points rather than duplicating logic.

3.4.2 Generation layer

Two interchangeable backends implement the generation contract behind a common interface, so that the choice of provider is a configuration flag rather than a code change. Both expose the same functions to call the model, extract the code from the response, and save the result, and both

Table 3.12: Repository layout and the concern each directory implements.

Directory	Contents
`generation/`	Provider backends and the CDK regeneration loop (`run_cdk_regen.py`).
`security_gate/`	The gate (`iac_security_gate.py`) and the scanner adapters under `security_gate/scanners/`.
`execution/`	The subprocess execution helper (`exec_code.py`).
`pipeline/`	Orchestration, credentials, and Phase 4 emitters.
`monitoring/`	CloudTrail, dashboards, the operational-loop function, and SSM registration.
`scripts/`	`run_cdk_pipeline.py` and `run_hybrid_pipeline.py`.
`ui/`	The Streamlit console and the multi-project registry.
`risk_scoring/`	Dataset preparation and model training.

centralise their default model identifiers while allowing an explicit override. Provider-specific error handling (for example, quota and throttling detection) is isolated behind the provider boundary, and credentials are read from explicit arguments first and then from the environment. The regeneration loop described in Algorithm 1 orchestrates these backends and persists the per-attempt artifacts.

3.4.3 Security gate and scanner adapters

The gate is implemented in `iac_security_gate.py`, which collects the synthesized templates, runs the heuristic checks and the scanner adapters, deduplicates the findings, computes the score of Equation 3.2, applies the decision function of Equation 3.3, and writes the gate report. Each scanner lives in its own adapter and returns a dictionary with at least a status and a

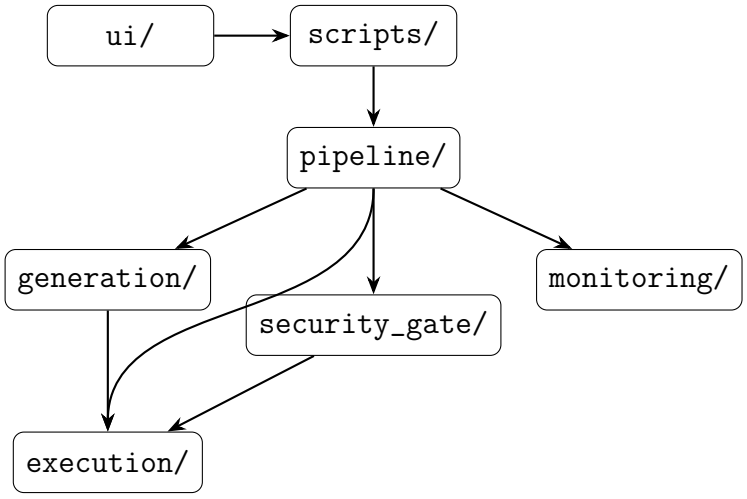

Figure 3.7: Runtime module dependencies. Entry points drive orchestration, which drives generation, evaluation, and monitoring; all external processes pass through the single execution helper.

message; no adapter ever raises, so a missing tool degrades to a well-defined status while the gate proceeds. Table 3.13 lists the adapters.

An abbreviated gate report is shown in Listing 3.1. The `components` object exposes the per-dimension contribution to the score, which makes the decision explainable, and the `scanner_status` object records which tools ran.

```
{
  "run_id": "cdk_20260614T093934Z",
  "timestamp": "2026-06-14T09:39:42+00:00",
  "score": 50,
  "decision": "review",
  "message": "Manual review required before deployment.",
  "components": {"severity": 30, "cost": 10, "aws_config": 10, "ml_risk":
   0},
  "findings": [
    {"severity": "high", "source": "checkov", "category": "sg_ssh_open",
     "resource_id": "MySG", "template": "App.template.json",
     "message": "Security group allows ingress on port 22 from 0.0.0.0/0"}
  ],
  "scanner_status": {"checkov": "ok", "cfn_lint": "ok",
                     "infracost": "ok", "aws_config": "ok", "ml_risk": "ok"
   }
```

Table 3.13: Scanner adapters and the signal each contributes.

Adapter	Signal
`checkov_adapter.py`	Policy-as-code findings on the synthesized templates.
`cfn_lint_adapter.py`	CloudFormation best-practice and validity findings.
`infracost_adapter.py`	Estimated monthly cost, used as the cost dimension.
`aws_config_adapter.py`	Count of non-compliant live-compliance rules.
`ansible_lint_adapter.py`	Findings on the on-premises playbooks.
`secret_scan_adapter.py`	Regular-expression secret detection (no external tool).
`ml_risk_adapter.py`	Logistic-regression probability of insecurity, converted to points.

```
}
```

Listing 3.1: Abbreviated gate report.

3.4.4 Orchestration and governance

Two command-line entry points drive the pipeline. The CDK-only entry point, `run_cdk_pipeline.py`, orchestrates bootstrap, synthesis, gating, diff, and deployment for the public side, and can optionally invoke the re-generation loop on a reject. The hybrid entry point, `run_hybrid_pipeline.py`, runs the same gate, risk-scoring, deployment, and observability workflow over both a bring-your-own CDK directory and a bring-your-own Ansible directory — with no code-generation step — gating each side independently with the same engine and writing a combined report. The orchestration routes all external commands through the execution backbone and honours the deployment-governance rules of Section 3.2.6. Listing 3.2 shows representative invocations.

```
# CDK-only: synth, gate, and deploy if the gate allows
```

```
python scripts/run_cdk_pipeline.py --project-dir generated_cdk --deploy

# Hybrid: gate both sides; scope the Ansible scan to files changed since
    HEAD~1
python scripts/run_hybrid_pipeline.py \
    --cdk-path examples/hybrid-demo/cdk \
    --ansible-path examples/hybrid-demo/ansible \
    --base-ref HEAD~1

# Read-only status of every gated target
python scripts/run_hybrid_pipeline.py --query-status
```

Listing 3.2: Representative pipeline invocations.

On the hybrid side, the Ansible scan can be scoped to the files changed since a given git reference, which limits analysis to the modified plays. The decision returned by each entry point is surfaced as a distinct process return code, so that both the console and any external continuous-integration system can react to *pass*, *review*, and *reject* without parsing free text.

Figure 3.8 traces the hybrid pipeline as a sequence of interactions between the orchestrator, the gate, the cloud and on-premises targets, and the monitoring layer, showing where the decision gates the deployment step on each side independently.

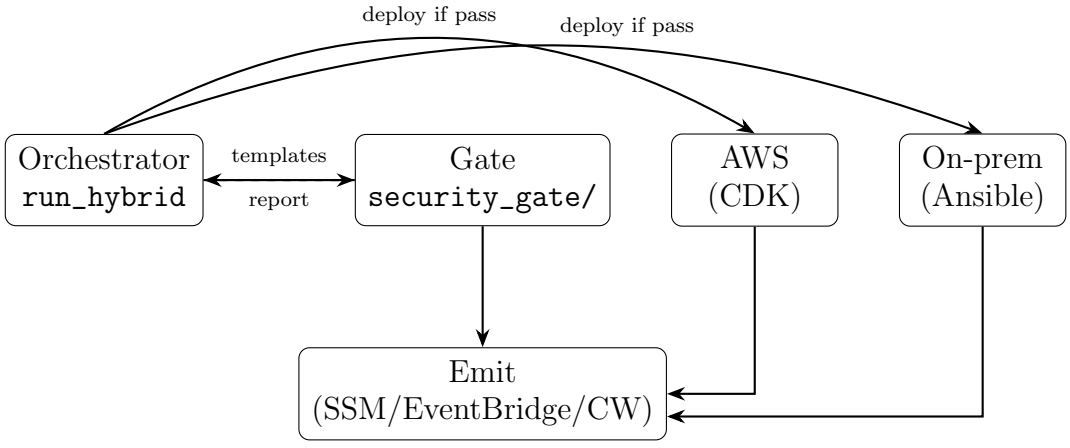

Figure 3.8: Hybrid pipeline interaction sequence. The orchestrator gates both sides with the same engine, deploys each side only on an accepting decision, and emits telemetry.

3.4.5 Execution backbone

All external commands (CDK, Ansible, and the scanners) run through a single subprocess helper in `exec_code.py`. The helper merges standard output and standard error to preserve the ordering of events, optionally streams output line by line for live rendering in the console, and returns the stable `{return_code, output}` contract that every caller relies on. Concentrating process execution in one place keeps the output-capture and error-propagation behaviour consistent across the whole system.

3.4.6 Operator console and multi-project registry

The Streamlit console is a thin orchestration layer over the command-line entry points: every privileged action is delegated to a subprocess with credentials injected through the environment, never as command-line arguments. The console presents the workflow as a sequence of tabs — settings, generation, gate review, plan/diff, deploy, and monitor — and renders the gate report, the decision, and the live monitoring reads. A multi-project registry allows several independent projects to coexist: the default project keeps a flat log layout for backward compatibility, while any other project owns a separate artifact directory so that gate reports, approvals, rejections, and monitoring history never mix between projects. The active project's log directory is exported to the subprocesses through an environment variable, which is how per-project isolation is enforced end to end.

Chapter 4

Implementation and Evaluation

4.1 Implementation Environment

The proposed SysSecOps process was implemented as a Python-based prototype operating across a hybrid cloud environment. The environment comprised a public-cloud domain and a private infrastructure domain connected through a common orchestration and security assessment workflow. The public-cloud domain provided programmable infrastructure, deployment, compliance information, and operational telemetry, whereas the private domain represented Linux-based on-premises systems managed through infrastructure automation. This arrangement enabled the same operational process and risk evaluation mechanism described in Chapter 3 to be exercised across heterogeneous infrastructure targets.

The implementation environment was organised into four functional groups: prototype execution, infrastructure provisioning, security assessment, and operational monitoring. Table 4.1 summarises the technologies used to realise each group. These technologies are implementation choices for evaluating the proposed process; the process itself is not restricted to these particular products.

Table 4.1: Technologies used in the implementation environment.

Environment group	**Technology**	**Role in the prototype**
Prototype execution	Python, command-line interface, and Streamlit	Implemented the process logic, pipeline entry points, and browser-based operator console.
AI-assisted generation	Amazon Bedrock and OpenRouter	Provided interchangeable language-model backends for generating and regenerating infrastructure code.
Public infrastructure	AWS CDK and AWS CloudFormation	Defined, synthesized, compared, and deployed resources in the public-cloud domain.
Private infrastructure	Ansible and Linux nodes	Represented and configured the on-premises domain of the hybrid environment.
Security assessment	Checkov, `cfn-lint`, `ansible-lint`, and a secret-detection component	Examined infrastructure artifacts for syntax errors, misconfigurations, insecure practices, and exposed secrets.
Contextual risk inputs	Infracost and AWS Config	Supplied estimated cost information and the live compliance state used by the risk-scoring mechanism.
Code-risk analysis	Bandit, Semgrep, and a logistic-regression model	Extracted static analysis features and estimated the probability that generated Python code contained detector-visible insecurity.
Monitoring and audit	AWS Systems Manager, Amazon CloudWatch, AWS CloudTrail, Amazon EventBridge, AWS Lambda, and Amazon SNS	Collected operational state, recorded activity, distributed events, and supported notification and remediation actions.

4.1.1 Prototype Execution Environment

Python was used as the principal implementation language for the prototype. The pipeline was accessible through command-line entry points and a Streamlit operator console. Both interfaces invoked the same orchestration logic, ensuring that an operation performed through the graphical

interface followed the same generation, assessment, approval, and deployment sequence as an operation initiated from the command line. External programs were invoked through a shared process-execution layer, which captured their return status and output in a consistent form for subsequent processing.

AI-assisted infrastructure generation was provided through interchangeable model providers. Amazon Bedrock represented a managed cloud-based model service, while OpenRouter provided an alternative hosted inference interface. The provider was selected through configuration without changing the remaining pipeline. Generated artifacts, synthesis output, scanner findings, risk components, and gate decisions were retained for each run, allowing an unsuccessful generation attempt to be examined and used as feedback for a subsequent attempt.

4.1.2 Hybrid Infrastructure Environment

The public part of the environment was hosted on Amazon Web Services. Infrastructure was expressed as an AWS Cloud Development Kit application written in Python and synthesized into AWS CloudFormation templates. The synthesized template, rather than only the high-level source code, was used as the principal public-cloud assessment artifact. This allowed the security gate to inspect the resource definitions and configuration values that would be submitted for deployment. Representative resources supported by the prototype included networking, compute, storage, and database services within a virtual private cloud.

The private part of the environment consisted of Linux nodes representing on-premises infrastructure. Their desired configuration was expressed through Ansible inventories and playbooks. Before execution, the playbooks were subjected to syntax and static analysis checks. Where operational monitoring across both domains was required, the private nodes were registered as managed instances through AWS Systems Manager hy-

brid activation. This provided a common management path for public-cloud instances and private Linux nodes while preserving their distinct deployment mechanisms.

Credentials and other sensitive configuration values were supplied to the prototype through the execution environment rather than embedded in generated infrastructure code or command-line examples. Deployment remained conditional on the security-gate result: an accepting result allowed the relevant provisioning mechanism to proceed, a review result required explicit approval, and a rejection prevented deployment.

4.1.3 Security Assessment and Risk Environment

The security assessment environment combined several complementary sources rather than depending on a single scanner. Checkov and `cfn-lint` evaluated synthesized CloudFormation templates, while `ansible-lint` evaluated the private-side playbooks. A built-in pattern-based component checked the submitted artifacts for embedded secrets. Their outputs were converted into the common finding representation defined by the proposed methodology before deduplication and scoring.

Two additional services supplied operational context to the assessment. Infracost estimated the financial effect of the proposed infrastructure, and AWS Config provided the compliance state of resources already operating in the public-cloud environment. Consequently, the gate decision reflected not only static misconfiguration findings but also cost and live compliance information.

The code-risk component was implemented as a logistic-regression classifier. Bandit and Semgrep were used to extract severity, confidence, and finding-count features from Python files. The trained classifier converted these features into a probability of detector-visible insecurity, which was then incorporated into the unified risk score. The persisted model and feature scaler were reused during evaluation so that inference followed the

same feature transformation as model training.

Each external assessment component was accessed through an adapter. An unavailable scanner, missing credential, or unsupported input was recorded as a component status instead of terminating the complete workflow. The resulting report therefore contained both the gate decision and the execution status of its contributing assessment sources, making the conditions under which each result was obtained explicit.

4.1.4 Monitoring and Audit Environment

After an accepted deployment, the environment used AWS Systems Manager to obtain managed instance information and support operational actions across the hybrid targets. Amazon CloudWatch collected gate metrics and operational observations, while AWS CloudTrail recorded public-cloud API activity for audit purposes. Amazon EventBridge distributed compliance and deployment events to the operational workflow. Event-driven processing and notification were supported through AWS Lambda and Amazon SNS, respectively.

The monitoring environment consumed the same target identifiers and gate records produced during assessment and deployment. This maintained traceability between a proposed infrastructure change, its security decision, the resulting deployment, and its subsequent operational state. It also provided the implementation basis for evaluating whether the proposed process could integrate pre-deployment assessment with post-deployment monitoring across the public and private domains.

4.2 Evaluation Methodology

The proposed SysSecOps process was evaluated through a controlled prototype case study comprising three complementary investigations. The first investigation examined the operational feasibility and execution reli-

ability of the complete pipeline. The second examined the applicability of the process to an existing on-premises virtual machine. The third examined the performance of the machine-learning component and the correctness of the unified risk-scoring mechanism. This design enabled the evaluation to address the operation of the prototype, its applicability within a hybrid environment, and its ability to transform heterogeneous security evidence into a reproducible deployment decision.

The evaluation focused on observable system behaviour rather than on the capabilities of any individual implementation tool. Evidence was collected from pipeline execution records, synthesized infrastructure artifacts, scanner outputs, risk reports, deployment records, target-state observations, and classifier predictions. The same scoring rules and decision thresholds defined in Chapter 3 were applied throughout the evaluation.

4.2.1 Research Questions

The evaluation was guided by the following research questions:

RQ1. Operational feasibility and reliability: To what extent can the proposed SysSecOps prototype execute the complete operational workflow, from infrastructure input or generation to assessment, decision, deployment, and monitoring, without unexpected pipeline failures?

RQ2. Hybrid infrastructure applicability: To what extent can the proposed process incorporate an existing on-premises Linux virtual machine through Ansible while maintaining common security assessment, deployment control, and operational traceability across the hybrid environment?

RQ3. Unified risk-evaluation capability: To what extent can the proposed risk-scoring mechanism integrate findings from multiple security tools and a logistic-regression code-risk estimate to distinguish

insecure code and produce consistent, interpretable deployment decisions?

Table 4.2 presents the relationship between the research questions, the corresponding evaluation methods, and the principal measurements.

Table 4.2: Relationship between the research questions and evaluation measures.

RQ	**Evaluation method**	**Collected evidence**	**Measures**
RQ1	Execution of representative end-to-end pipeline scenarios	Stage results, gate reports, deployment records, monitoring records, and execution logs	Completion rate, stage completion, unexpected failures, and decision enforcement
RQ2	Integration and configuration of an existing Linux virtual machine through Ansible	Connectivity results, playbook analysis, execution records, gate reports, and target-state observations	Integration success, configuration success, policy enforcement, and state verification
RQ3	Held-out classifier evaluation and controlled risk-decision cases	True labels, predicted probabilities, confusion matrix, component scores, deduplicated findings, and gate decisions	Accuracy, precision, recall, F1-score, ROC–AUC, score correctness, and decision consistency

4.2.2 Evaluation Design

The unit of analysis for RQ1 was one complete pipeline run. A run began when an infrastructure request or existing project was submitted to the prototype and ended when the process produced a persisted terminal decision. Where the decision permitted deployment, the run additionally included deployment and post-deployment status collection. For RQ2, the unit of analysis was one pipeline-controlled Ansible operation against an existing virtual machine. For RQ3, an individual Python file constituted the unit of analysis for classifier evaluation, while one controlled combination of risk inputs constituted the unit of analysis for gate-decision evaluation.

The evaluation varied three principal conditions: the risk characteristics of the submitted infrastructure, the deployment target, and the availability of optional assessment components. The observed dependent variables included completion status, stage outcome, gate decision, deployment outcome, resulting target state, predicted code-risk probability, and agreement between independently calculated and reported risk scores.

A security rejection was not classified as a pipeline failure. A run that identified an unacceptable change, produced a valid *reject* decision, and prevented deployment represented correct execution of the security process. An unexpected failure was defined as an unhandled exception, premature pipeline termination, absent or malformed decision report, incorrect transition between stages, or deployment that violated the gate and approval policy.

4.2.3 End-to-End Pipeline Evaluation

RQ1 was investigated by executing the prototype across representative scenarios that exercised the principal paths through the operational workflow. Each run was observed at the following checkpoints:

1. acceptance or generation of the infrastructure source;
2. synthesis of the public-cloud template or syntax validation of the private-side playbook;
3. execution of the configured assessment components;
4. normalisation and deduplication of the collected findings;
5. calculation and persistence of the component scores, total score, and gate decision;
6. enforcement of the deployment and approval policy; and

7. publication of the decision and deployed-target state to the monitoring and audit layer.

Four scenario classes were included. A low-risk change exercised the *pass* path, a moderate-risk change exercised the manual *review* path, and a deliberately insecure change exercised the *reject* path. A degraded-execution scenario was also included by making an optional assessment dependency or credential unavailable. This scenario examined whether the corresponding component returned an explicit unavailable or skipped status without causing an unhandled termination of the complete pipeline. The degraded scenario was analysed separately because successful continuation with a missing assessment source provides less security evidence than a fully assessed run.

For n pipeline runs, the end-to-end completion rate was calculated as

$$\text{Completion Rate} = \frac{n_{\text{terminal}}}{n} \times 100\%, \tag{4.1}$$

where n_{terminal} denotes the number of runs that reached a valid and persisted terminal decision without an unexpected failure. Stage-level completion was reported in addition to the aggregate rate to identify whether failures occurred during generation, validation, assessment, scoring, deployment, or monitoring. Policy enforcement was verified by confirming that a rejected change was not deployed and that a review decision could proceed only after an explicit approval record had been created.

4.2.4 Existing On-Premises Virtual Machine Evaluation

RQ2 examined the use of the proposed process with infrastructure that existed before the pipeline run and was located outside the public-cloud provisioning workflow. The target was a Linux virtual machine represented in an Ansible inventory and configured through an Ansible playbook. The

virtual machine retained its existing operating environment and was therefore treated as the private-side target of the hybrid case study.

The evaluation procedure comprised six stages. First, network reachability and authenticated Ansible access to the target were verified. Second, the inventory and playbook were submitted to syntax validation and static security assessment. Third, the resulting findings were normalised and processed by the same risk engine and decision thresholds used for public-cloud changes. Fourth, playbook execution was permitted only when the gate decision and approval state satisfied the deployment policy. Fifth, the requested configuration state was verified directly on the virtual machine. Finally, the target identity, gate decision, execution result, and post-execution state were recorded for operational traceability.

Successful integration required all of the following conditions: authenticated access to the existing target, successful analysis of the playbook, production of a valid gate decision, enforcement of that decision before execution, successful application of an allowed configuration, and verification of the requested target state. Where the playbook described an idempotent operation, a subsequent execution was used to determine whether the desired state remained stable without unintended configuration changes.

Observed failures were classified as connectivity, authentication, validation, assessment, gate-enforcement, configuration-execution, or target-verification failures. This classification separated failures caused by the external infrastructure from those caused by the implementation of the proposed process.

4.2.5 Risk-Scoring Evaluation

RQ3 was examined through two linked evaluations. The first measured the ability of the logistic-regression component to discriminate between labelled insecure and reference secure Python files. The second examined whether the complete gate correctly aggregated the machine-learning result

with the remaining risk components and produced the decision specified by the scoring policy.

Dataset Construction

The evaluation corpus contained a positive class and a negative reference class. The positive class was obtained from the SecurityEval dataset, which contains Python vulnerability examples associated with Common Weakness Enumeration categories and was developed for evaluating the security of machine-learning-based code-generation techniques [27]. Each selected vulnerability example was stored as an individual Python file and assigned label 1.

The negative class was sampled from the source trees of maintained open-source Python projects, including `click`, `rich`, and `typer`. Each selected file was assigned label 0. These files constituted a *reference secure* class rather than a formally verified secure class. Their use in maintained software projects provided practical negative examples, but did not establish the absence of every possible vulnerability.

To improve comparability between the two classes, package initialisers and test files were excluded. Files containing fewer than 20 or more than 700 non-blank, non-comment lines were also excluded. These criteria reduced the influence of trivial snippets, tests, and unusually large files on the learned model. The number of samples, class distribution, and originating project or dataset were retained as dataset metadata. The final corpus was balanced as far as practical to limit majority-class bias.

Feature Extraction and Model Training

Each file was analysed independently using Bandit and Semgrep. The findings were reduced to the eleven-dimensional feature vector defined in Table 3.10. The features represented Bandit severity counts, Bandit confidence counts, Semgrep severity counts, and the total number of findings

returned by each analyser.

The corpus was divided into training and test partitions using a stratified 80/20 split. Stratification preserved the class distribution in both partitions. A feature scaler was fitted exclusively on the training partition and was then applied unchanged to the held-out test partition. The logistic-regression classifier was trained with balanced class weights, and its output for each file was the estimated probability of membership in the insecure class. Fixing the data split and preprocessing configuration enabled the experiment to be reproduced under the same conditions.

Classifier Evaluation Measures

Classifier performance was assessed using the confusion matrix, accuracy, precision, recall, F1-score, and the area under the receiver operating characteristic curve (ROC–AUC). Let TP, TN, FP, and FN denote the numbers of true positives, true negatives, false positives, and false negatives, respectively. The principal measures were calculated as

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \tag{4.2}$$

$$\text{Precision} = \frac{TP}{TP + FP}, \tag{4.3}$$

$$\text{Recall} = \frac{TP}{TP + FN}, \tag{4.4}$$

$$F_1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}. \tag{4.5}$$

Accuracy represented overall predictive correctness, whereas precision measured the proportion of predicted insecure files that belonged to the positive class. Recall measured the proportion of labelled insecure files identified by the model. Recall was particularly relevant to the gate because a false-negative result reduced the learned risk contribution assigned to insecure code. The F1-score summarised the balance between precision

and recall, while ROC–AUC measured the ability of the model to rank positive samples above negative samples across probability thresholds.

A scanner-only rule provided the baseline for comparison with the logistic-regression classifier. The baseline classified a file as insecure when at least one configured security finding was reported. This comparison determined whether learning a relationship among the scanner features provided discriminatory value beyond treating the presence of any finding as a positive classification.

Unified Score and Decision Evaluation

The complete risk-scoring mechanism was evaluated using controlled cases containing different combinations of deduplicated static findings, estimated infrastructure cost, live compliance state, and machine-learning probability. For each case, the expected component contributions and total score were calculated independently using Equation 3.2. The independently calculated values were then compared with the values recorded in the gate report.

Cases were constructed below, at, and above the decision boundaries defined in Equation 3.3. This boundary-oriented design exercised the *pass*, *review*, and *reject* outcomes and examined whether the implemented comparisons treated threshold values consistently. Additional cases contained equivalent findings produced by more than one scanner to determine whether the deduplication procedure prevented repeated counting of one underlying issue.

The gate evaluation considered three properties:

- **Aggregation correctness**: the reported score equalled the sum of the deduplicated severity, cost, compliance, and machine-learning contributions.
- **Decision consistency**: the reported decision corresponded to the score interval defined by the configured thresholds.

- **Interpretability**: the report identified the contributing findings, component values, scanner execution states, total score, and resulting decision.

The classifier evaluation and the controlled gate cases addressed different aspects of RQ3. The classifier measures assessed the discriminatory performance of the learned code-risk component. The gate cases assessed whether that component and the other risk signals were combined correctly and transparently to govern deployment.

4.2.6 Data Collection and Analysis

Evidence from each pipeline scenario was collected from the machine-readable artifacts produced by the prototype. These artifacts included stage return states, scanner execution states, normalised and deduplicated findings, component scores, final gate decisions, approval records, deployment results, and monitoring records. For the on-premises scenario, Ansible execution output and direct target-state observations were also retained. For the classifier evaluation, the true label, predicted class, predicted probability, and source of each test sample were recorded.

Quantitative results were summarised using counts, rates, classification measures, and agreement between expected and observed decisions. Qualitative inspection was limited to failure classification and assessment of whether a gate report contained sufficient information to trace a decision to its contributing evidence. Results were organised by research question so that conclusions could be derived from the corresponding evaluation evidence rather than from individual tool outputs in isolation.

4.2.7 Threats to Validity

The evaluation was conducted as a prototype case study and therefore has limitations in external validity. The selected public-cloud resources,

on-premises virtual machine, and infrastructure scenarios do not represent every hybrid cloud configuration. Network conditions, service availability, credentials, and the state of external services may also affect execution independently of the proposed process. These environmental states were recorded to distinguish transient infrastructure failures from pipeline defects.

The construction of the machine-learning dataset introduces a threat to construct validity. SecurityEval provides labelled vulnerable examples, whereas files sampled from maintained open-source projects are not formally proven to be free of vulnerabilities. The negative class may therefore contain label noise. In addition, the features extracted by Bandit and Semgrep represent only weaknesses observable through their configured rule sets. Consequently, the classifier was evaluated as an estimator of *detector-visible insecurity*, rather than as a general detector of all semantic software vulnerabilities.

The fixed risk weights and decision thresholds constitute an operational policy rather than a universally optimal definition of risk. The controlled cases can establish that the prototype implements this policy consistently, but cannot demonstrate that the same thresholds are appropriate for every organisation. Evaluation in additional hybrid environments and calibration using organisation-specific risk preferences would be required to support broader generalisation.

4.3 Evaluation Results

This section is organised according to the three research questions defined in Section 4.2.1. Quantitative results are available for the learned code-risk component associated with RQ3. The end-to-end pipeline experiment and the existing virtual-machine experiment have not yet been executed; therefore, their subsections contain reporting templates rather than empirical findings. This distinction prevents incomplete observations

from being presented as evaluation evidence.

4.3.1 RQ1: Operational Feasibility and Reliability

This experiment concerns execution of the complete SysSecOps pipeline for the pass, review, reject, and degraded-component scenarios described in Section 4.2.3. Its reporting structure distinguishes a valid security rejection from an unexpected execution failure. A rejected change is classified as a completed run when the pipeline produces a valid gate report and correctly prevents deployment.

Table 4.3 defines the reporting structure for the experiment. The completion rate is calculated using Equation 4.1, while stage-level records identify the location and cause of unsuccessful runs.

Table 4.3: Result structure for the end-to-end pipeline evaluation.

Measure	**Result**	**Evidence source**
Total pipeline runs	—	Execution records
Runs reaching a valid terminal decision	—	Gate reports
End-to-end completion rate	—	Calculated from run records
Pass, review, and reject outcomes	—	Persisted decision records
Unexpected pipeline failures	—	Stage results and execution logs
Rejected changes incorrectly deployed	—	Gate and deployment records
Review cases executed without approval	—	Approval and deployment records
Degraded-component runs reaching a terminal state	—	Scanner status and gate reports

The completed table is intended to show whether each submitted case reached the expected terminal state, whether the gate decision was enforced, and whether unavailable optional components were represented ex-

plicitly instead of causing an unhandled termination. No empirical RQ1 result is reported before these observations are collected.

4.3.2 RQ2: Hybrid Infrastructure Applicability

This experiment concerns integration of an existing Linux virtual machine through the Ansible path of the prototype. Evidence is collected at the connectivity, assessment, decision, execution, state-verification, and operational-recording stages. Table 4.4 defines the corresponding reporting structure.

Table 4.4: Result structure for the existing virtual-machine evaluation.

Evaluation checkpoint	**Result**	**Verification evidence**
Network reachability	—	Connectivity test
Authenticated Ansible access	—	Ansible connection result
Playbook syntax validation	—	Validation output
Security assessment and gate decision	—	Scanner output and gate report
Enforcement of the gate before execution	—	Decision and execution records
Application of the requested configuration	—	Ansible execution result
Verification of the target state	—	Direct observation of the virtual machine
Repeated-run state consistency	—	Subsequent Ansible execution result
Operational and audit record creation	—	Status and audit records

The completed checkpoint records provide separate evidence for infrastructure access, security assessment, policy enforcement, configuration execution, and post-execution verification. This separation allows connectivity and authentication failures to be distinguished from failures within

the proposed security process. No empirical RQ2 result is reported before the experiment is executed.

4.3.3 RQ3: Unified Risk-Evaluation Capability

The available RQ3 evidence comprises the held-out performance of the logistic-regression model and an examination of its fitted coefficients. This analysis assesses whether features obtained from multiple static analysers provide a useful learned signal for the unified risk-scoring mechanism. The reported results concern detector-visible insecure code and do not represent the detection of every semantic vulnerability.

Dataset Composition

The corpus was constructed from two classes of Python files. Approximately 80 positive examples were selected from SecurityEval. Vulnerability categories that could not be represented by the configured Bandit and Semgrep features were excluded from the model corpus. The negative reference class contained approximately 80 files sampled from maintained Python projects, including `click`, `rich`, and `typer`. To reduce the difference in scale between the short SecurityEval examples and project source files, negative examples were restricted to files containing more than 20 and fewer than 300 non-blank, non-comment lines.

The negative examples are referred to as *reference secure* files rather than formally verified secure files. Maintained project code can still contain unknown vulnerabilities, and the assigned negative labels therefore represent a practical reference assumption. Similarly, excluding vulnerability categories outside the observable scope of the selected analysers narrows the interpretation of the results. The reported performance applies to the retained, detector-visible subset and is not an estimate of performance over the complete SecurityEval dataset.

The held-out test partition contained 31 files: 16 negative samples and

15 positive samples. The near-balanced distribution limited the influence of class frequency on the reported accuracy.

Classification Performance

Table 4.5 presents the aggregate performance of the logistic-regression classifier on the held-out test partition. The classifier achieved an accuracy of 0.8710 and a ROC–AUC of 0.8792. Precision, recall, and F1-score for the positive class were all 0.8667.

Table 4.5: Logistic-regression performance on the held-out test partition.

Measure	**Result**
Accuracy	0.8710
Precision	0.8667
Recall	0.8667
F1-score	0.8667
ROC–AUC	0.8792
Test samples	31

The confusion matrix is presented in Table 4.6. Of the 16 negative samples, 14 were classified correctly and two were incorrectly classified as insecure. Of the 15 positive samples, 13 were identified correctly and two were incorrectly classified as reference secure. The model therefore produced four errors across the 31 held-out samples.

Table 4.6: Confusion matrix for the held-out test partition.

	Predicted negative	**Predicted positive**
Actual negative	14	2
Actual positive	2	13

Table 4.7 gives the class-specific results. The negative class obtained precision, recall, and F1-score values of approximately 0.88, while the positive class obtained approximately 0.87 for all three measures. The macro

and weighted averages were both approximately 0.87, indicating that performance was similar across the two classes in the held-out partition.

Table 4.7: Class-specific classification results.

Class	**Precision**	**Recall**	**F1-score**	**Support**
Negative (0)	0.88	0.88	0.88	16
Positive (1)	0.87	0.87	0.87	15
Macro average	0.87	0.87	0.87	31
Weighted average	0.87	0.87	0.87	31

The equal positive-class precision and recall resulted from the symmetric number of false-positive and false-negative predictions. From an operational perspective, the two false negatives are particularly relevant because they represent labelled insecure files that would receive a lower learned risk contribution. The two false positives have a different consequence: they may increase the score of reference negative code and cause unnecessary review, but they do not directly permit an insecure file to bypass the learned component.

Analysis of Model Coefficients

Table 4.8 presents the fitted logistic-regression coefficients in descending order. Because the features were standardised before training, coefficient magnitudes provide an indication of their relative influence within this fitted model. A positive coefficient indicates that an increase in the corresponding feature increases the estimated log-odds of the insecure class, with the remaining features held constant.

The total number of Semgrep findings had the largest positive coefficient (1.136296), followed by the number of medium-confidence Bandit findings (1.003949) and high-severity Semgrep findings (0.914679). This ordering indicates that the fitted model relied most strongly on the overall Semgrep signal and on medium-to-high-strength findings from the two

Table 4.8: Coefficients of the trained logistic-regression model.

Feature	Coefficient
total_semgrep	1.136296
bandit_conf_medium	1.003949
semgrep_high	0.914679
semgrep_medium	0.720708
total_bandit	0.610932
bandit_medium	0.454592
bandit_high	0.435620
bandit_low	0.261740
bandit_conf_high	0.138912
bandit_conf_low	0.083852
semgrep_low	0.067123

analysers. Low-severity and low-confidence features had smaller positive coefficients and therefore contributed less strongly to the predicted probability.

These coefficients must be interpreted with caution. The total-finding features are mathematically related to the corresponding severity counts, which introduces correlated predictors. The coefficients describe the behaviour of the fitted model and do not establish that one feature has a causal relationship with software insecurity.

Per-File Risk Records

In addition to the aggregate classification measures, the corpus is reprocessed through the trained component to obtain a predicted probability and risk contribution for each file. The resulting machine-readable artifact is archived with the project repository for inspection and reproducibility. Each record contains the sample identifier, dataset source, assigned label, extracted feature values, predicted probability, predicted class, and learned risk contribution. When the complete set of gate inputs is available, the record also contains the component scores, total score, and resulting deployment decision.

Table 4.9 defines the fields retained in this supplementary artifact. The

aggregate distribution of per-file probabilities and risk scores is reported after the re-execution records have been finalised.

Table 4.9: Fields retained in the supplementary per-file risk artifact.

Field	**Description**
Sample identifier	Unique identifier or relative path of the evaluated Python file
Source	SecurityEval or the originating reference project
Assigned label	Positive (1) or negative reference (0) class
Feature vector	Bandit and Semgrep counts supplied to the trained model
Model output	Predicted probability, predicted class, and learned risk contribution
Gate output	Component scores, total score, and decision when all gate inputs are available

Table 4.10 defines the reporting structure for the controlled gate-decision experiment. The experiment evaluates aggregation correctness, decision boundaries, and cross-tool deduplication after the per-file risk records have been finalised.

Table 4.10: Result structure for the controlled gate-decision evaluation.

Case	**Expected**	**Observed**	**Agreement**
Score below or equal to pass threshold	Pass	—	—
Score immediately above pass threshold	Review	—	—
Score equal to review upper boundary	Review	—	—
Score above reject threshold	Reject	—	—
Duplicate finding reported by multiple tools	Counted once	—	—

Summary of Risk-Model Findings

Within the detector-visible and filtered evaluation corpus, the logistic-regression component achieved an accuracy of 0.8710, an F1-score of 0.8667,

and a ROC–AUC of 0.8792. The similar class-specific measures indicate that the result was not obtained by favouring only one class. The coefficient analysis further shows that the model incorporated signals from both Bandit and Semgrep, with the strongest fitted influence associated with Semgrep finding volume, medium-confidence Bandit findings, and high-severity Semgrep findings.

The observed performance indicates that a learned multi-tool signal can contribute discriminatory information to the proposed risk-scoring mechanism within the evaluated corpus. It does not establish complete vulnerability detection, because the corpus excludes vulnerability categories outside the configured analysers' detection scope and uses maintained project files as a reference negative class. Evaluation of the complete deployment-decision mechanism additionally requires the per-file gate records and the controlled score-boundary cases defined in Section 4.2.5.

Chapter 5

Discussion

Chapter 6

Conclusion

Danh mục công trình của tác giả

1. Tạp chí ABC
2. Tạp chí XYZ

Reference

[1] M. Armbrust et al., "Above the clouds: A berkeley view of cloud computing," Tech. Rep. UCB/EECS-2009-28, Feb. 2009. [Online]. Available: http://www2.eecs.berkeley.edu/Pubs/TechRpts/2009/EECS-2009-28.html

[2] B. Varghese and R. Buyya, "Next generation cloud computing: New trends and research directions," *Future Generation Computer Systems*, vol. 79, pp. 849–861, 2022.

[3] Flexera, *2026 state of the cloud report*, https://www.flexera.com/blog/finops/flexera-2026-state-of-the-cloud-report-the-convergence-of-cloud-and-value/, Accessed: 2026-06-05, 2026.

[4] P. Mell and T. Grance, "The nist definition of cloud computing," National Institute of Standards and Technology, Tech. Rep. Special Publication 800-145, 2011. DOI: 10.6028/NIST.SP.800-145 [Online]. Available: https://doi.org/10.6028/NIST.SP.800-145

[5] F. Liu et al., "Nist cloud computing reference architecture," National Institute of Standards and Technology, Tech. Rep. Special Publication 500-292, 2011. DOI: 10.6028/NIST.SP.500-292 [Online]. Available: https://doi.org/10.6028/NIST.SP.500-292

[6] M. Armbrust et al., "A view of cloud computing," *Communications of the ACM*, vol. 53, no. 4, pp. 50–58, 2010. DOI: 10.1145/1721654.1721672

[7] Flexera, *2025 state of the cloud report*, https://info.flexera.com/CM-REPORT-State-of-the-Cloud, Accessed: 2026-07-17, 2025.

[8] HashiCorp, *2025 state of cloud strategy survey*, https://www.hashicorp.com/state-of-the-cloud, Accessed: 2026-07-17, 2025.

[9] K. Dodson, M. Souppaya, and K. Scarfone, "Secure software development framework (ssdf) version 1.1," National Institute of Standards and Technology, Tech. Rep. Special Publication 800-218, 2022. DOI: 10.6028/NIST.SP.800-218

[10] H. Myrbakken and R. Colomo-Palacios, "Devsecops: A multivocal literature review," in *International Conference on Software Process Improvement and Capability Determination (SPICE)*, ser. Lecture Notes in Business Information Processing, vol. 290, Springer, 2017, pp. 17–29. DOI: 10.1007/978-3-319-67383-7_2

[11] R. N. Rajapakse, M. Zahedi, M. A. Babar, and H. Shen, "Challenges and solutions when adopting devsecops: A systematic review," *ACM Computing Surveys*, vol. 55, no. 11, pp. 1–45, 2023. DOI: 10.1145/3579635

[12] X. Zhao, T. Clear, and R. Lal, "Identifying the primary dimensions of devsecops: A multi-vocal literature review," *Journal of Systems and Software*, vol. 214, p. 112 063, 2024, ISSN: 0164-1212. DOI: 10.1016/j.jss.2024.112063

[13] L. Bass, I. Weber, and L. Zhu, *DevOps: A Software Architect's Perspective*. Addison-Wesley Professional, 2015, ISBN: 9780134049847.

[14] J. Humble and D. Farley, *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley Professional, 2010, ISBN: 9780321601919.

[15] OWASP Foundation, *OWASP Software Assurance Maturity Model (SAMM) Version 2.1*, https://owaspsamm.org/, 2024.

[16] P. Mell, K. Scarfone, and S. Romanosky, "Guide to enterprise patch management planning: Preventive maintenance for technology," National Institute of Standards and Technology, Tech. Rep. NIST Special Publication 800-40 Revision 4, 2022. DOI: 10.6028/NIST.SP.800-40r4

[17] Forum of Incident Response and Security Teams (FIRST), *Common Vulnerability Scoring System Version 4.0: Specification Document*, https://www.first.org/cvss/v4.0/specification-document, 2023.

[18] C. Maplestone et al., *Stakeholder-Specific Vulnerability Categorization (SSVC) Version 2.0*, https://certcc.github.io/SSVC/, 2023.

[19] M. Bozorgi, L. K. Saul, S. Savage, and G. M. Voelker, "Beyond heuristics: Learning to classify vulnerabilities and predict exploits," in *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2010, pp. 105–114. DOI: 10.1145/1835804.1835821

[20] Y. Shin and L. Williams, "Can traditional fault prediction models be used for vulnerability prediction?" *Empirical Software Engineering*, vol. 18, no. 1, pp. 25–59, 2013. DOI: 10.1007/s10664-011-9193-7

[21] F. Rahman, D. Posnett, A. Hindle, and P. Devanbu, "Bugcache for inspections: Hit or miss?" *IEEE Transactions on Software Engineering*, 2019.

[22] A. Vaswani et al., "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 5998–6008.

[23] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, et al., "Language models are few-shot learners," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020, pp. 1877–1901.

[24] M. Chen, J. Tworek, H. Jun, Q. Yuan, et al., *Evaluating large language models trained on code*, 2021. arXiv: 2107.03374 [cs.LG].

[25] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, "Asleep at the keyboard? assessing the security of GitHub Copilot's code contributions," in *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*, 2022, pp. 754–768. DOI: 10.1109/SP46214.2022.9833571

[26] N. Perry, M. Srivastava, D. Kumar, and D. Boneh, "Do users write more insecure code with AI assistants?" In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2023, pp. 2785–2799. DOI: 10.1145/3576915.3623157

[27] M. L. Siddiq and J. C. S. Santos, "SecurityEval dataset: Mining vulnerability examples to evaluate machine learning-based code generation techniques," in *Proceedings of the 1st International Workshop on Mining Software Repositories Applications for Privacy and Security*, 2022. DOI: 10.1145/3549035.3561184

Appendix A

Ngữ pháp tiếng Việt

Đây là phụ lục.

Appendix B

Ngữ pháp tiếng Nôm

Đây là phụ lục 2.